DESIGN AND IMPLEMENTATION OF AI-POWERED RETAIL CHATBOT

Kaviya Shankar
Student ID: @00757645
Aug- 2025

A dissertation submitted in Partial Fulfilment of the requirements for the degree of Master of Data Science at University of Salford

Title

DESIGN AND IMPLEMENTATION OF AI-POWERED RETAIL CHATBOT

By: Kaviya Shankar

@00757645

Supervisor: Arunachalam Sundaram

School of Science, Engineering and Environment

University of Salford, Manchester, United Kingdom

Academic Year: 2024-25

Abstract

This dissertation examines the design and implementation of an AI-powered retail chatbot employing Generative Pre-trained Transformers (GPT) to enhance customer interaction and administrative decision-making within the retail sector. The system adopts a dual-portal architecture comprising a customer-facing chatbot capable of managing order-related queries—including tracking, cancellations, refunds, and product inquiries—and an administrative interface that provides monitoring, segmentation, and data-driven insights through Power BI integration. The implementation was realised using a modern technology stack, incorporating React.js, Node.js, Python, Microsoft SQL Server, and the OpenAI API. Structured prompt engineering and customer relationship management (CRM) integration were utilised to ensure contextually coherent and business-compliant outputs. The system was evaluated through functional, usability, and performance testing, demonstrating high levels of accuracy, reduced dependency on human intervention, and effective support for managerial analytics. Notable limitations include latency associated with API calls, the potential for generative inconsistencies, and operational cost considerations. The findings affirm that GPT-powered chatbots represent a viable and scalable solution for retail service automation, with recommendations for future work including multilingual support, voice interaction, payment integration, and advanced analytics to further strengthen customer engagement and organisational value. (Project recording link has added in appendices)

Acknowledgement

I wish to extend my sincere gratitude to my supervisor, Mr. Arunachalam Sundaram, for his continuous guidance, insightful feedback, and encouragement throughout the development of this dissertation. His academic expertise and constructive suggestions were invaluable in refining both the direction and the quality of this work.

I would also like to acknowledge the University of Salford, School of Science, Engineering and Environment, for providing the academic resources and supportive environment that enabled the successful completion of this study.

Finally, I am grateful to all individuals who contributed, either directly or indirectly, to the completion of this research. Their support and assistance have been instrumental in the realization of this dissertation.

Table of Contents

CHAPTER – 1 10
Introduction 10
1.1. General Overview 10
1.2. Scope 10
1.3. Objectives 11
1.4. Methodology 12
1.5. Tools and Technologies 13
1.6. Data Analysis Approach 14
1.7. Summary 14
CHAPTER – 2 16
Literature Review 16
2.1 Introduction 16
2.2 Chatbots in Retail 16
2.3 Evolution of Conversational AI 16
2.4 Generative Pre-trained Transformers (GPT) 17
2.5 Integration of GPT into Retail Chatbots 17
2.6 Synthesis of Related Literature 18
2.1 Limitation of GPT-Powered Chatbots 19
2.2 Summary 20
Chapter – 3 21
System Design and Architecture 21
3.1 Introduction 21
3.2 Architecture Overview 21
3.3 Client Layer 22
3.4 Web Browser Layer 24
3.5 Backend Layer 25
3.6 External Service Layer (OpenAI GPT API) 26
3.7 Data Flow Summary 27
Chapter – 4 28
Implementation 28
4.1 Customer Panel 28
4.1.1 System Architecture 28
4.1.2 Authentication Module 29
4.1.3 Frontend Implementation 32

4.1.4 Backend Architecture 34
4.1.5 LLM & NLP 36
4.1.6 Structured Prompt Engineering for Reliable Chatbot Integration 41
4.1.7 CRM – Integrated Data Management 43
4.1.8 Functional Modules 44
4.1.9 Operational Dataflow & Process Architecture 45
4.1.10 Deployment & Environment 46
4.1.11 Rationale for OpenAI Integration 47
4.1.12 Testing & Evaluation 48
4.1.13 Key Considerations 50
4.1.14 Summary 51
4.2 Admin Panel 52
4.2.1 Architecture Overview 52
4.2.2 Frontend Implementation 53
4.2.3 Backend Implementation 55
4.2.4 Database Implementation 57
4.2.5 Data-Driven Functionality of Interface Screens 59
4.2.6 Authentication and Validation 60
4.2.7 Charting and CSV Delivery 60
4.2.8 Implementation Flow 61
4.2.9 Deployment and Environment 62
4.2.10 Key Points 62
4.3 Interactive Chatbot and Analytics Power BI Dashboard 63
4.3.1 Data Sources and Data Model 63
4.3.2 Relationship Diagram 64
4.3.3 High-Level Implementation Framework 64
4.3.4 Technical Implementation Workflow 65
4.3.5 Comparative Summary of Implementation Approaches 67
4.3.6 Dashboard Components 67
4.3.7 Analytical Value 69
4.3.8 Integration in the Admin Panel 70
4.3.9 Summary & Key Points 71
Chapter – 5 72
Critical Evaluation 72
5.1. Evaluation Against Objectives 72
5.2. Review of the plan and Deviations 73

5.3. Lessons Learned 74
5.4. Limitations of the study 75
Chapter – 6 76
Future Works & Conclusion 76
6.1. Future Recommendations 76
6.2. Comparative Summary of Future Features 77
6.3. Conclusion 77
Chapter – 7 78
Reference 78
Appendices 80

List of Figures:

Figure 1: System Architecture 22
Figure 2: Repository Structure of Customer Panel 29
Figure 3: Customer Login Screen 30
Figure 4: Login in Progress 31
Figure 5: Chat Interface with Customer Info 32
Figure 6: Config.py 35
Figure 7: LLM & NLP Flow Chart 39
Figure 8: gpt_service.py 41
Figure 9: Customer Panel Flowchart 46
Figure 10: Repository Structure of Admin Panel 53
Figure 11 Login Bar Chart 54
Figure 12 Login Heatmap 54
Figure 13: dbo.Orders 57
Figure 14 dbo.CustomerRequests 57
Figure 15: dbo.LoginEvents 58
Figure 16: db_service.py 58
Figure 17 Top Products by Order Frequency 59
Figure 18: Admin Panel Flow Chart 61
Figure 19: Logical Relationship Diagram of DB 64
Figure 20: Retail Chat Analytics Dashboard 69
Figure 21 Order Status Enquiry. 80
Figure 22 Order Cancellation 80
Figure 23 Product Inquiry 81
Figure 24 Exchange Request 81
Figure 25 Customer Representative Request 82
Figure 26: Usage Tracking. 82
Figure 27: Purchase Credit 83
Figure 28: Admin Login Page 83
Figure 29: Admin Credentials Validations 84
Figure 30: Admin Banner 84
Figure 31: Navigation Bar 84
Figure 32: Login Bar Chart 85
Figure 33: Login Line Chart 85
Figure 34: Logins Heatmap 86
Figure 35: IPs & Countries 86
Figure 36: Phone Login Counts 86
Figure 37: Top Products by Order Frequency 87
Figure 38: Recent Logins 87
Figure 39: All Orders 88

List of Tables

Table 1: Intent Mapping (Reference) 40
Table 2: Acceptance Criteria & Results 49
Table 3: Distinctions Between High-Level Framework & Technical Framework 67
Table 4: Comparison of Proposed Future Features 77

List of Abbreviations

1) **AI** – Artificial Intelligence
2) **API** – Application Programming Interface
3) **AWS** – Amazon Web Services
4) **BERT** – Bidirectional Encoder Representations from Transformers
5) **CORS** – Cross-Origin Resource Sharing
6) **CRM** – Customer Relationship Management
7) **CRUD** – Create, Read, Update, Delete
8) **CSS** – Cascading Style Sheet
9) **CSV** – Comma-Separated Values
10) **DAX** – Data Analysis Expression
11) **DB** – Database
12) **DDQN** – Double Deep Q-Network
13) **ENV** – Environment Variable
14) **ERD** – Entity Relationship Diagram
15) **FCR** – First Contact Resolution
16) **FK** – Foreign Key
17) **GDPR** – General Data Protection Regulation
18) **GPT** – Generative Pre-trained Transformer
19) **HTML** – Hypertext Markup Language
20) **HTTPS** – Hypertext Transfer Protocol Secure
21) **IP** – Internet Protocol
22) **JS** – JavaScript
23) **JSON** – JavaScript Object Notation
24) **KPI** – Key Performance Indicator
25) **LLM** – Large Language Models
26) **NLG** – Natural Language Generation
27) **NLP** – Natural Language Processing
28) **NPM** – Node Package Manager
29) **ODBC** – Open Database Connectivity
30) **PK** – Primary Key

31) **Power BI** – Business Intelligence

32) **RAG** – Retrieval-Augmented Generation

33) **REST** – Representational State Transfer

34) **SQL** – Structured Query Language

35) **SSMS** – SQL Server Management Studio

36) **UI** – User Interface

CHAPTER – 1

Introduction

1.1. General Overview

The retail industry's rapid digital transition has drastically changed how businesses engage with their clientele. A major breakthrough that has allowed businesses to provide immediate assistance, increase service availability, and boost consumer engagement is chatbots. Although they work well for pre-programmed queries, traditional rule-based chatbots frequently lack adaptability and a natural conversational flow. Generative Pre-trained Transformers (GPT), which use sophisticated natural language processing (NLP) to produce more intelligent, context-aware interactions, have been integrated as a result.

Building a solid system that serves administrators and consumers is the main goal of the current project, Design and Implementation of a GPT-Powered Retail Chatbot for Enhanced Customer Interaction. While administrators have access to traffic metrics, customer segmentation, heatmaps, and reporting dashboards, customers may use the chatbot to help with order details, cancellations, refunds, and general questions. This dual-portal strategy guarantees that the system offers useful insights for store management in addition to improving customer interaction.

1.2. Scope

Designing and implementing a retail chatbot driven by GPT that facilitates order monitoring, refunds, cancellations, and general questions through intelligent, context-aware dialogues is the goal of this project. The system offers an administrator gateway for traffic monitoring, customer segmentation, and Power BI analytics in addition to the consumer-facing functionality. Through the integration of contemporary technologies like Python, SQL, React.js, Node.js, and the OpenAI API, the project aims to provide a scalable and effective solution that benefits administrators and users alike.

1.3. Objectives

1. **To design and implement a customer-facing chatbot interface powered by GPT, complemented with rule-based prompts:** This objective focuses on creating an intelligent yet reliable conversational system that can assist customers with a wide range of queries such as product searches, order tracking, and service-related actions including refunds and cancellations. The integration of GPT ensures context-aware and natural responses, while the rule-based framework provides consistent handling of structured tasks.
2. **To develop a secure and efficient administrative interface for monitoring and management:** The system aims to equip administrators with tools to oversee user traffic, validate login details, and manage customer information such as IDs, contact numbers, and country data. This ensures that customer engagement is not only efficient but also well-organized and securely monitored.
3. **To incorporate advanced reporting and visualization features for data-driven insights:** This involves implementing charts, reports, and heatmaps that provide real-time visibility into customer interactions and order patterns. By presenting complex data in an accessible visual format, the system supports administrators in identifying trends, evaluating customer behaviour, and making informed business decisions.
4. **To ensure data security, privacy, and system reliability:** Since the chatbot handles sensitive customer information, the project prioritises secure authentication, input validation, and compliance with data protection principles. This objective aims to maintain trust and safeguard both customer and business data.
5. **To evaluate the effectiveness of the system through functionality testing and limited user-based validation:** This objective ensures that the implemented features — including chatbot interactions for refunds, cancellations, and order tracking on the customer side, as well as traffic monitoring, reporting, and analytics on the admin side — perform as intended. The evaluation involves verifying correctness, reliability, and user satisfaction to confirm that the system meets its intended goals.
6. **To analyse system performance and propose directions for future enhancement:** Beyond the implementation, the project seeks to reflect critically on its strengths and limitations. Recommendations such as incorporating multi-language support, integration with

payment gateways, and deeper customer behaviour analytics will be proposed to ensure scalability and relevance in future retail applications.

1.4. Methodology

The methodology adopted for this project followed a structured and systematic approach to ensure that the chatbot system addressed both customer-facing functionality and administrative requirements. Each stage of the process was carefully planned and executed, beginning with the identification of requirements and concluding with deployment and evaluation.

The process began with ***Requirement Analysis***, where the functional and non-functional requirements of the system were identified. On the customer side, the chatbot needed to handle order-related queries such as tracking, cancellations, and refunds, as well as provide general support in a conversational manner. On the administrative side, the requirement was to create a secure portal that could monitor customer traffic, segment customer information, generate reports, and provide data visualization for decision-making. This phase also involved analysing existing solutions to understand the limitations of traditional rule-based chatbots, which guided the design of a GPT-based system.

In the ***Implementation phase***, the system was developed using a modern and efficient technology stack. The customer-facing chatbot was integrated with the OpenAI API through Python, enabling intelligent, context-aware conversations. The frontend was built with React.js, HTML, and CSS to ensure responsiveness and usability across devices. The backend, developed using Node.js, managed the interaction between the chatbot, database, and administrative portal. Data storage and management were handled using Microsoft SQL Server, which maintained structured customer and order information. Together, these technologies provided a seamless integration of the chatbot with both customer and admin functionalities.

The Analytics Integration phase was critical for the administrative portal. Here, customer interaction data and order logs were processed and visualized using Power BI. This allowed the generation of real-time charts, reports, and heatmaps, providing insights into customer behaviour, order patterns, and chatbot performance. By linking SQL data to Power BI,

administrators could monitor trends and make informed decisions to enhance customer service strategies.

Once implemented, the system underwent ***Testing and Validation.*** Multiple tests were conducted, including functional testing (to verify chatbot responses and portal functions), usability testing (to ensure smooth interaction for customers and admins), and performance testing (to assess response times, data handling, and stability under load). The chatbot was evaluated in different simulated retail scenarios to check its accuracy and reliability in handling real-world queries.

Finally, the project reached ***the Deployment and Evaluation phase***, where the system was hosted in a secure local environment with HTTPS protocols to ensure safe communication. The deployed system was then evaluated against the project's objectives, confirming its ability to provide natural customer interactions while offering administrators meaningful insights and analytics for decision-making.

Important Steps in the Approach:

- Analysing requirements involves determining administrative and consumer needs.
- System Design: Creation of an architecture with two portals (administrator and client).
- Implementation: React.js, Node.js, Python, and SQL integration in a chatbot driven by GPT.
- Analytics Integration: Use Power BI to visualize reports, heatmaps, and interactions.
- Verifying chatbot accuracy, usability, and integration effectiveness through testing and validation.
- Deployment and Evaluation: HTTPS hosting and practical performance validation.

1.5. Tools and Technologies

1. **Programming Languages:**
    a) *Python (chatbot logic, GPT integration, backend functions*
    b) *SQL (database management)*
    c) *React.js, HTML, CSS (frontend design and user interface)*
    d) *Node.js (server-side logic and API management)*

2. **Tools and Frameworks:**
    a) *Visual Studio Code – for coding and debugging across frontend and backend components.*
    b) *OpenAI API – GPT-powered conversational intelligence.*
    c) *Microsoft SQL Server Management Studio – for structured data storage and retrieval.*
    d) *Power BI – for customer interaction analysis, report generation, and visualization.*

1.6. Data Analysis Approach

In this study, Microsoft Power BI was employed as the primary platform for analysing customer interactions and generating actionable insights. Customer activity data—including *order requests, cancellations, refunds, and chat transcripts*—was systematically collected and stored in SQL Server, ensuring secure management and reliable retrieval of information. The raw data was then processed, cleansed, and transformed manually into structured formats suitable for analysis, between the SQL Server backend and Power BI's visualisation environment.

Using this framework, the processed data was represented through a variety of visual outputs, including charts, heatmaps, and interactive dashboards. These tools enabled administrators to identify behavioural patterns, track customer engagement, and evaluate the overall performance of the chatbot system in real time. By providing both granular and aggregated perspectives, the analytical process not only ensured methodological rigour but also facilitated evidence-based decision-making. Ultimately, this integrated approach bridged the gap between raw operational data and strategic business intelligence, thereby enhancing the effectiveness of retail operations and contributing to the optimisation of customer service processes.

1.7. Summary

This chapter has provided a comprehensive overview of the project, outlining its overarching aims and specific objectives, while also describing the technologies, tools, and methodologies employed throughout the development process. The discussion emphasised the systematic

approach adopted to ensure technical reliability, user-centred design, and practical applicability within the intended context.

Building on this foundation, the subsequent chapter presents a critical review of the existing literature, with a particular focus on the evolution of chatbot technologies and the growing significance of Generative Pre-trained Transformers (GPT) in advancing conversational artificial intelligence. The review further examines previous studies related to the application of chatbots within the retail sector, highlighting both their potential to enhance customer engagement and the limitations that continue to constrain their effectiveness.

Through this examination, key research gaps are identified, including the need for more contextually adaptive chatbot systems, the integration of advanced natural language understanding in consumer-facing applications, and a deeper exploration of user perceptions regarding trust and usability. Addressing these gaps provides the justification for the present study, which aims to contribute to both the academic discourse and practical development of retail chatbot solutions by leveraging recent advancements in GPT-based conversational models.

CHAPTER – 2

Literature Review

2.1 Introduction

Conversational AI has advanced rapidly through breakthroughs in natural language processing (NLP) and large language models (LLMs) such as GPT. From scripted agents to complex interfaces, chatbots in the retail industry may now help customers with questions, provide recommendations, and facilitate transactional support (Vaswani et al., 2017; OpenAI, 2023).

2.2 Chatbots in Retail

Retail-focused chatbots are now used in a wide range of commerce sectors in addition to providing basic customer support. They increasingly perform tasks such as answering queries, guiding customers through checkout processes, and facilitating order tracking and after-sales service. At the same time, some retail applications employ them for product discovery and recommendations, illustrating their versatility across the customer journey (Gentsch, 2019; Huang & Rust, 2021).

2.3 Evolution of Conversational AI

The development of conversational AI has progressed through several distinct phases. Early systems such as ELIZA in the 1960s were based on simple pattern-matching techniques, providing rigid and repetitive responses with little contextual understanding (Weizenbaum, 1966). These early models demonstrated the potential of human-computer conversation but were limited by their rule-based nature.

With advancements in machine learning, more flexible systems emerged. Sequence-to-Sequence (Seq2Seq) models allowed chatbots to learn conversational patterns from large datasets, improving fluency and adaptability (Sutskever, Vinyals, & Le, 2014). However, these models still struggled with long-term context retention and often generated inconsistent answers.

The introduction of transformer architectures (Vaswani et al., 2017) marked a significant breakthrough. By leveraging self-attention mechanisms, transformers improved the handling of long-range dependencies in language, enabling more coherent and contextually accurate responses. This paved the way for large language models (LLMs).

Modern LLMs such as GPT-3, with 175 billion parameters, introduced the ability to perform zero-shot and few-shot learning, allowing them to generalize effectively without task-specific retraining (OpenAI, 2020).

2.4 Generative Pre-trained Transformers (GPT)

The Generative Pre-Trained Transformer (GPT) family represents a major advancement in natural language processing, building on the transformer architecture introduced by Vaswani et al. (2017). GPT models are trained through unsupervised pre-training, where they learn general language patterns from large-scale text corpora, followed by fine-tuning on specific tasks when needed (Radford et al., 2019). This approach enables the models to develop a deep understanding of syntax, semantics, and context.

A defining feature of GPT models is their ability to maintain contextual coherence across multiple conversation turns. Unlike earlier sequence models that struggled with long-range dependencies, GPT can interpret user input in context, generate natural responses, and adapt to different conversational tones (Brown et al., 2020). This makes them particularly effective for dynamic, real-time retail interactions where customer queries vary widely in complexity.

GPT also excels in natural language generation (NLG), producing fluent, human-like responses that can manage diverse retail tasks such as handling order inquiries, providing product recommendations, or assisting with refund and cancellation policies. Importantly, GPT models can deal with unexpected or incomplete inputs, offering greater flexibility than rule-based or template-driven chatbots (AIMultiple, 2024).

2.5 Integration of GPT into Retail Chatbots

The integration of GPT into retail chatbots typically follows a multi-layered architecture that connects the frontend, backend, and database with the GPT model through secure APIs.

Frontend: Frameworks such as React.js are commonly employed to build interactive and responsive interfaces where customers can input their queries. This ensures usability and smooth communication between users and the chatbot system (PLANEKS, 2024).

Backend: The backend acts as the middleware, connecting the frontend with the GPT model. Technologies such as Node.js and Python manage API requests and process responses from GPT before sending them back to the user (GeeksforGeeks, 2024).

Database Layer: Retail applications often integrate with SQL-based databases to manage customer details, product information, and order histories. This allows chatbots to provide personalized and context-aware responses by retrieving relevant records during interactions (Medium, 2024).

Security Layer: Since customer interactions involve sensitive information, a security layer is incorporated to safeguard data. HTTPS protocols, authentication, and API key management ensure secure communication between all components.

Supporting Technologies: Integration is streamlined by tools such as the official OpenAI Python library, which simplifies API communication. Middleware frameworks can optionally be used to extend capabilities, though they are not essential to all implementations.

Advanced Architectures: Novel approaches such as Retail-GPT demonstrate how Retrieval-Augmented Generation (RAG) can enhance retail chatbots. By combining GPT's generative capabilities with retrieval from structured databases, these systems deliver more accurate recommendations and improved customer support (AIMultiple, 2024).

2.6 Synthesis of Related Literature

The quantifiable influence of chatbots on retail performance is highlighted by recent empirical research. According to Rahevar et al. (2024), chatbots driven by AI increased the accuracy of product selection by about 48% and significantly increased customer happiness, retention, and trust. In the FMCG industry, Zia and Alotaibi (2024) found a direct correlation between customer happiness and responsiveness and privacy issues.

Khneyzer (2024) emphasized from a managerial standpoint that AI chatbots in CRM systems lower expenses while enhancing productivity, client retention, and satisfaction. Isa et al.

(2024) benchmarked chatbot pipelines employing components like BERT, DDQN, and GPT-2 for technical evaluation, offering metrics for goal-oriented performance.

Dey and Bhaumik (2022) investigated trust, happiness, and continuity expectations from a behavioral standpoint, defining important elements in retail chatbot client adoption. Lastly, Singh (2025) provided a thorough analysis of evaluation techniques for chatbots with LLM integration, including information on testing methodologies and performance metrics.

2.1 Limitation of GPT-Powered Chatbots

Despite their advancements, GPT-powered chatbots face several challenges that must be considered in retail applications.

Hallucinations: GPT models sometimes generate outputs that appear fluent but are factually incorrect. For retail, this could result in wrong product details, pricing errors, or misinformation about return policies, which can harm customer trust (AP News, 2024). Developers often attempt to mitigate this through fine-tuning or retrieval-augmented generation, but the problem is not fully resolved.

Latency Issues: Since GPT queries are processed via cloud-based APIs, response delays can occur, particularly during high traffic. In customer service, even small delays can lead to dissatisfaction as users expect fast and seamless interactions (GeeksforGeeks, 2024).

High Operational Costs: The cost of using powerful GPT models is considerable, especially for large-scale retailers handling thousands of queries per day. Subscription fees and API call charges can make it challenging for smaller businesses to adopt such solutions sustainably (Barron's, 2024). Some organizations mitigate this by using smaller models for routine queries and reserving GPT for complex requests.

Limited Business Rule Control: Unlike rule-based systems where responses are predefined, GPT dynamically generates answers. This flexibility makes enforcing strict business policies, compliance standards, or domain-specific rules more difficult (AIMultiple, 2024). In retail, this raises concerns about consistency in promotions, legal disclaimers, or regulatory compliance.

Ethical and Privacy Concerns: Since GPT systems process sensitive customer data, issues of privacy, data storage, and ethical usage arise. Businesses must ensure compliance with data

protection regulations (e.g., GDPR) while avoiding biases in AI-generated responses that may negatively affect customer experience.

2.2 Summary

From rule-based systems to transformer-powered LLMs, this analysis covers the development of retail chatbots. GPT has brought about notable advancements in conversational fluency, scalability, and personalization. Python is commonly used alongside JavaScript frameworks in modern chatbot stacks. Despite the obvious advantages of GPT integration, problems with hallucinations, latency, cost, and rule enforcement still exist. While empirical research demonstrates that retail chatbots increase customer satisfaction, engagement, and retention, it also emphasizes the significance of meticulous planning, assessment, and integration tactics.

Chapter – 3

System Design and Architecture

3.1 Introduction

System design, which establishes the solution's structural and functional blueprint, is an essential phase in the software development lifecycle. The architecture of the retail chatbot system was thoughtfully created to capitalize on AI-driven conversational intelligence while supporting both administrative and customer-facing functionalities. A React-based interface, a FastAPI backend, a relational database running SQL Server, and the OpenAI GPT API for natural language processing are all integrated into the design.

According to Bass, Clements, and Kazman (2021), the system's layered modular architecture ensures scalability, maintainability, and separation of responsibilities. By using this strategy, the architecture guarantees a dependable pipeline for managing client demands like order tracking, refunds, and cancellations, facilitates effective communication between administrators and consumers, and delivers precise real-time responses. Additionally, the system was built with extensibility in mind, enabling the addition of monitoring and sophisticated analytics features in the future (Garlan & Shaw, 2017).

3.2 Architecture Overview

The system architecture *(as Show in figure 1)* is composed of multiple layers, including the client interface, backend middleware, database, and external GPT service. Each component is integrated to support seamless after-sales interactions. As outlined in Section 2.5, retail chatbot architectures are typically organized around a layered approach that spans the frontend, backend, database, and security layers. In alignment with this model, the system employs a **React.js frontend** to provide an interactive customer interface, a **Node.js and Python backend** to manage API communication with GPT, and an **SQL database** for storing order details and transaction histories. Data security is ensured through HTTPS protocols and API key management. This design demonstrates direct consistency with the layered integration patterns discussed in the literature.

- ***Client Layer (Local Host – User/Admin):*** Customers and administrators access the system via a local host connection secured with HTTPS.
- ***Web Browser Layer:*** Provides the customer chatbot interface and the admin dashboard.
- ***Backend Layer:*** Handles application logic, rule-based intent recognition, database operations, and GPT integration.
- ***External Service Layer (OpenAI GPT API):*** This layer provides natural language processing and contextual understanding. As discussed in Section 2.3 on the evolution of conversational AI, earlier models such as ELIZA and Seq2Seq were constrained by rigid pattern-matching and limited ability to retain long-term context, often resulting in inconsistent responses. The adoption of transformer-based models, and later GPT, addressed these shortcomings by introducing self-attention and large-scale pre-training, enabling more coherent and context-aware interactions. Building on this progression, the system integrates GPT to ensure reliable, fluent, and contextually appropriate support for after-sales services.

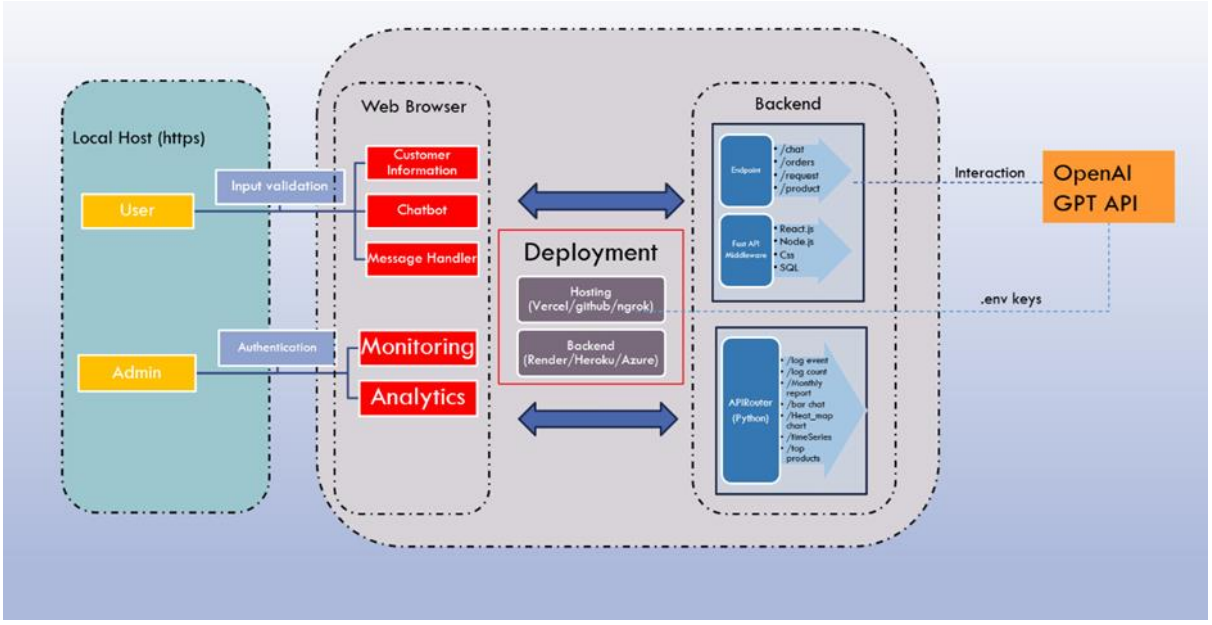


Figure 1: System Architecture

3.3 Client Layer

The client layer provides the entry point for users of the system. It consists of two categories of actors: customers and administrators. Each role is mapped to specific tasks and restricted

privileges. In alignment with the literature on retail chatbots (see Section 2.2), which highlights their potential to span the entire customer journey, this implementation narrows its focus to the **after-sales phase**. Accordingly, the customer-facing chatbot is designed to assist with order status checks, cancellations, and refund queries, while administrators are provided with interfaces to monitor and manage these interactions.

3.3.1 User Interaction

Customers are authenticated through a simple login process using their **phone number**, which serves as the primary identifier in the database. Once authenticated, the system retrieves the most recent order from the database and presents it to the customer. The chatbot then serves as the primary interface for user interaction, allowing them to:

- Cancel or modify an order.
- Request a refund.
- Check delivery status.
- Escalate to human support.

This design reduces dependency on human customer service agents and provides a **self-service support channel**, aligning with trends in digital customer service automation (Shawar & Atwell, 2007).

3.3.2 Admin Interaction

The **administrator role** provides oversight and control. Admins must authenticate before accessing the dashboard, which contains two key sections:

- **Order Management:** Displays all customer orders with their current status.
- **Request Management:** Lists all logged customer service requests (e.g., cancellations, refunds, escalations).

The admin panel supports operational decision-making by enabling staff to monitor customer issues in real time. Research suggests that integrating customer-facing AI tools with human oversight improves customer trust and service efficiency (Lankton, McKnight, & Tripp, 2015).

3.4 Web Browser Layer

The web browser layer serves as the presentation tier and is implemented using React.js. It renders dynamic, responsive, and interactive interfaces for both customers and administrators.

3.4.1 Customer Interference

The **customer information panel** displays retrieved details, including **name, phone, order ID, product, and current status.** By presenting these details transparently, the system fosters trust between the customer and the retail service (Nielsen, 2012).

3.4.2 Chatbot Interface

The **chatbot** forms the core of customer interaction. It supports:

- **Text-based communication** via an input box.
- **Voice-based communication** using browser speech recognition APIs.
- **Quick Action Buttons** for common tasks such as Cancel Order and Check Status.

Such multimodal interaction enhances usability by catering to different user preferences, which is consistent with modern principles of human–computer interaction (Dix, Finlay, Abowd, & Beale, 2004).

3.4.3 Message handler

The message handler ensures smooth communication between frontend and backend. It packages messages into structured JSON payloads, forwards them to the backend API, and processes the response. GPT-generated responses are tagged (TO_USER for customer-facing and TO_CRM for internal notes), and the handler displays only the customer-facing messages. This separation ensures clarity in communication and prevents internal data leakage (Bass et al., 2021).

3.4.4 Monitoring & Analysis

For administrators, the system provides monitoring and analytics dashboards that consolidate both technical and customer-facing data. On the monitoring side, key metrics such as GPT response times, API latency, and failure rates are tracked to ensure reliable system

performance. On the analytics side, aggregated customer request data—covering cancellations, refunds, and product complaints—is visualized to highlight recurring issues and bottlenecks. These dashboards support data-driven decision-making, enabling managers to improve service delivery and enhance customer satisfaction by identifying frequent pain points and optimizing operations (Provost & Fawcett, 2013; Chen, Chiang, & Storey, 2012).

3.5 Backend Layer

The backend layer is implemented using **FastAPI**, a high-performance Python web framework. It orchestrates communication between the frontend, database, and GPT API.

3.5.1 Deployment

The backend exposes RESTful API endpoints:

- */order* – fetches order details using phone number.
- */chat* – processes chatbot interactions and integrates with GPT.
- */orders* – retrieves all orders for admin view.
- */requests* – retrieves all logged requests.

REST APIs ensure stateless communication, which enhances scalability and fault tolerance (Richardson, 2018).

3.5.2 Core Backend Modules

The backend contains three core modules:

- **Chat Handler:** Processes user queries with rule-based logic for cancellation, refund, and escalation. Non-matching queries are sent to GPT.
- **Database Handler:** Provides CRUD operations for the *Orders* and *Requests* tables via ODBC.
- **GPT Service Module:** Manages conversations, sends queries to GPT, and filters responses into *TO_USER* and *TO_CRM*.

This modular approach is consistent with microservices principles, ensuring that individual components can evolve without impacting the entire system (Richardson, 2018).

3.5.3 ***Database Design***

The system employs a **SQL Server relational database** to ensure reliable data persistence and efficient retrieval.

- ***Order Table***

  Stores all customer order details:

  - ✓ *OrderID (primary key)*
  - ✓ *CustomerName*
  - ✓ *Phone*
  - ✓ *Status (e.g., Placed, Shipped, Delivered, Cancelled)*
  - ✓ *Product*

- ***Request Table***

  Logs customer service actions:

  - ✓ RequestID (primary key)
  - ✓ OrderID (foreign key)
  - ✓ RequestType (e.g., Refund, Cancel, Exchange, Return, Escalation)
  - ✓ RequestDate

- ***Data Flow***

  Customer login retrieves order from *Orders*.

  Cancel/refund requests update order status and insert into *Requests.*

  Admins query consolidated views for decision-making.

Relational models are widely adopted in CRM systems for their ability to handle structured, interrelated data (Coronel & Morris, 2016).

3.6 External Service Layer (OpenAI GPT API)

The OpenAI GPT API is integrated to provide **natural language understanding.**

- When customer queries do not match rule-based intents, they are forwarded to GPT.
- GPT responses are structured into *TO_USER* and *TO_CRM*.
- Only *TO_USER* outputs are displayed to the customer, while *TO_CRM* notes are logged for administrators.

This integration significantly reduces the workload of human support agents, as conversational AI can resolve many common issues autonomously (Shawar & Atwell, 2007).

3.7 Data Flow Summary

The end-to-end workflow can be summarized as follows:

1) Customer logs in with phone → backend retrieves order from database.
2) Customers interact via chatbot (text, voice, or quick buttons).
3) Backend applies rule-based intent recognition; unresolved queries sent to GPT.
4) GPT returns tagged responses → *TO_USER* shown to customer, *TO_CRM* logged.
5) Admins monitor orders and requests via the dashboard.

This flow ensures **real-time customer interaction, database consistency,** and **administrative oversight.**

Chapter – 4

Implementation

Building on the architecture and design discussed in Chapter 3, this chapter focuses on the system's implementation. While the earlier chapter highlighted the conceptual framework and workflows, here the emphasis is on translating those designs into actual software modules, database integrations, and interfaces. The implementation covers the customer interface, user journey, frontend and backend architecture, prompt engineering, CRM integration, security and privacy features, accessibility, testing, and ethical considerations. This process follows established practices in software engineering, which stress structured modular development and clear mapping between design and code (Sommerville, 2016).

4.1 Customer Panel

4.1.1 System Architecture

The following will explain the flow of frontend &backend repository structure of customer panel *(As shown in Figure 2)*

Multi-tier Architecture: The Customer Panel follows a three-layered design: frontend, backend, and persistence/AI services. This ensures modularity, maintainability, and scalability.

Frontend (React.js + CSS): Built with modular components such as *Login.js, ChatWindow.js*, and *CRMPanel.js.* Each handles a specific responsibility, making the UI responsive, professional, and accessible.

Backend (Python FastAPI): Provides REST endpoints for authentication, order queries, and chatbot integration. It bridges frontend requests with the database and AI service.

Persistence Layer (SQL Server): Contains structured tables like *Orders, CustomerRequests*, and *LoginEvents.* This setup guarantees reliable data transactions, traceability, and fast query execution.

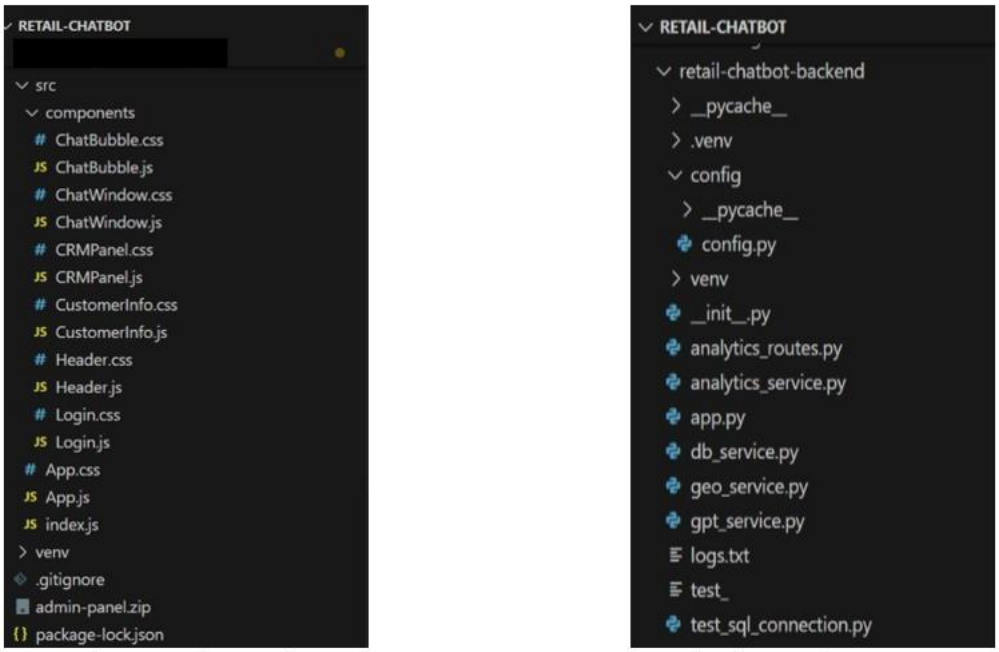


Figure 2: Repository Structure of Customer Panel

AI Integration (OpenAI GPT- 3.5): Adds conversational intelligence with enforced rules for polite, clear, and predictable responses. Prompt engineering ensures the chatbot maintains compliance with business logic.

Independent Evolution: Each system layer can evolve separately while communicating effectively, producing a stable and extensible architecture.

4.1.2 Authentication Module

A. *Login Screen*

Simplified Authentication: Phone-based login reduces complexity and improves accessibility by allowing customers to log in using only their registered phone number *(as shown in figure 3)*. This eliminates traditional password fatigue and ensures faster entry for all users.

Implementation: The *Login.js* component validates numeric inputs and enforces formatting checks. Combined with CSS styling, the interface delivers a modern, clean, and intuitive login page.

Backend Role: Submitted phone numbers are verified through a *POST /auth/check-phone* API call. SQL Server cross-references the input with existing customer data, ensuring secure access control.

User Experience: Visual feedback such as a loading spinner ("*Checking…*") reassures customers during the process. This keeps users engaged and prevents duplicate submissions.

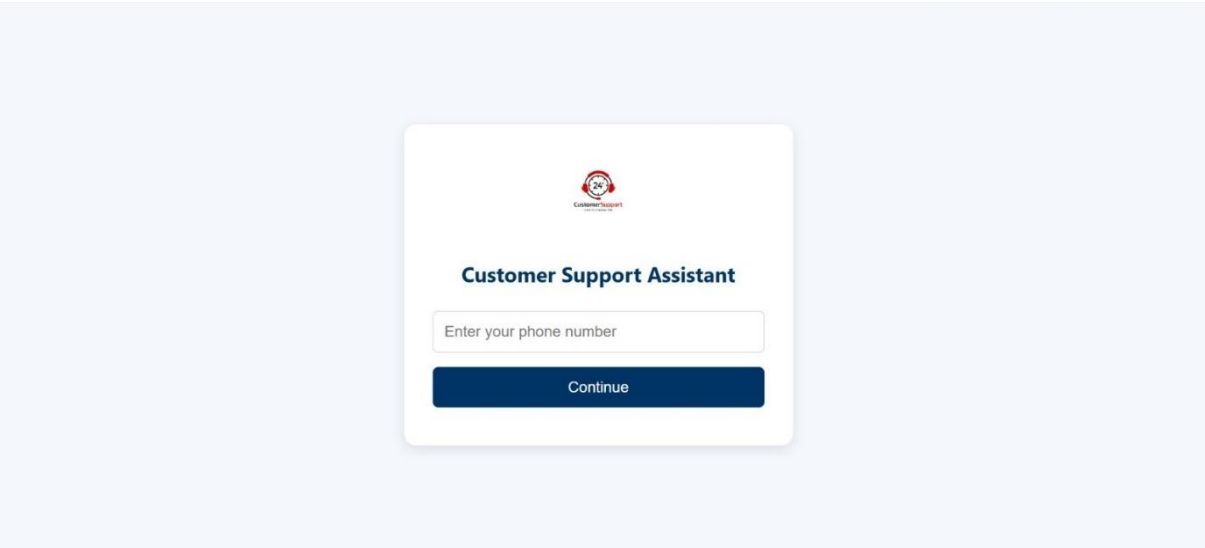


Figure 3: Customer Login Screen

B. *Validation*

Loading Feedback: The system communicates clearly with users by displaying a verification state, ensuring transparency during login attempts. *(as shown in figure 4)*

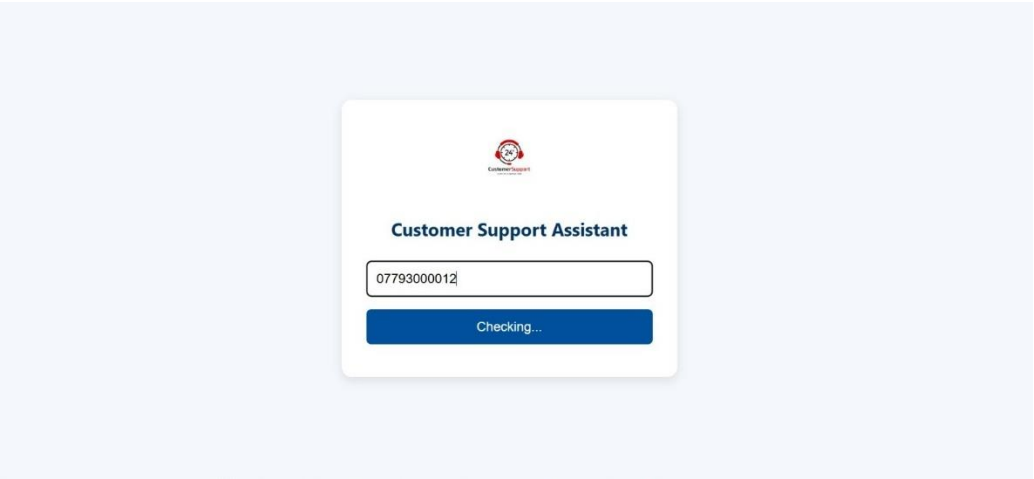


Figure 4: Login in Progress

Backend Verification: Using parameterized SQL queries, the system validates phone numbers and fetches associated active orders, ensuring accuracy and preventing SQL injection vulnerabilities. This validation strategy aligns with the ethical and privacy concerns highlighted in Section 2.6 of the Literature Review.

Audit Logging: Login attempts are stored in the LoginEvents table, capturing metadata such as timestamps, phone number, IP address, and user-agent, providing accountability and security.

```
"""
SELECT TOP 1 OrderID, CustomerName, Phone, Status, Product, Color
FROM dbo.Orders
WHERE Phone = ?
ORDER BY OrderID DESC
"""
```

C. *<u>Profile Display</u>*

Data Binding: On successful validation, the chatbot retrieves customer details and binds them to the session. Displaying these details reinforces system reliability.

Interface Integration: Profile data is shown via *CustomerInfo.js* and *CRMPanel.js*, giving customers confidence that their identity and order details are accurately recognized.

Chatbot Activation: A structured greeting from GPT, shaped by prompt engineering, initiates conversation, ensuring consistency and professionalism. *(as shown in figure 5)*

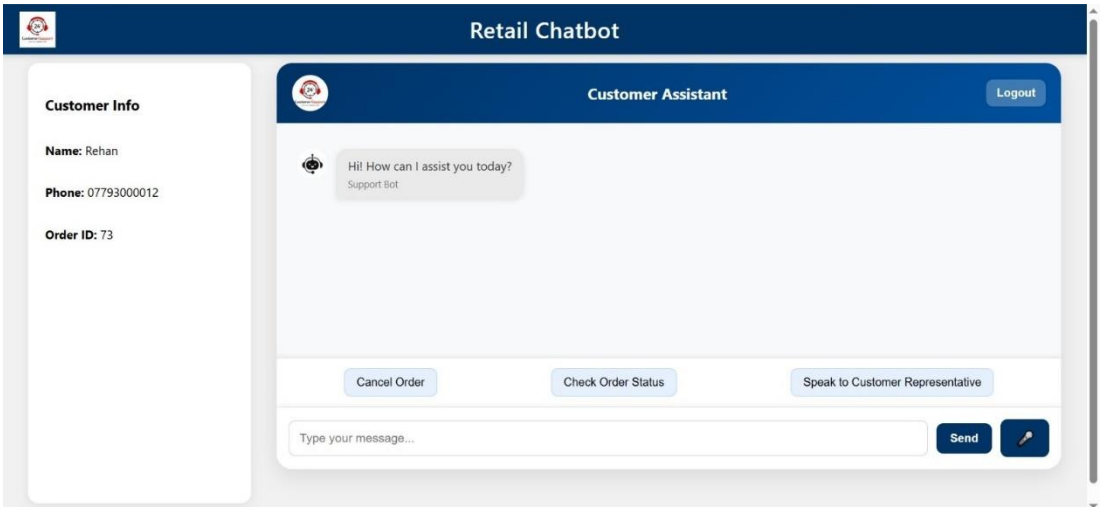


Figure 5: Chat Interface with Customer Info

4.1.3 Frontend Implementation

The front-end implementation represents the customer-facing layer of the chatbot system, combining both design and functionality to provide an intuitive interface. It was developed using React for modularity and reusability, supported by HTML5 and CSS3 to deliver structure, presentation, and styling. This layer ensures that users can seamlessly navigate from login to interaction with the chatbot while maintaining a professional and accessible user experience.

A. <u>Technology Stack</u>

The technology stack for the front-end of the chatbot is centred on a combination of React.js, HTML5, CSS3, and Node.js, each serving a critical role. React.js forms the foundation by offering a modular, component-driven architecture that allows developers to build interfaces that are both reusable and maintainable.

HTML5 and CSS3 complement this by providing the structural and stylistic layers of the application. HTML5 ensures semantic organization of content, which not only supports accessibility but also facilitates search engine optimization. CSS3, in turn, controls the visual design, making it possible to implement responsive layouts, consistent branding, and interactive effects that enhance usability.

Node.js operates in the background to support the build pipeline and manage dependencies essential for the React ecosystem. It allows for fast compilation, package management through npm, and streamlined integration of third-party libraries, all of which accelerate

development cycles. Together, these technologies form a robust and flexible stack that delivers a professional, reliable, and user-friendly front-end experience.

B. Component Structure

The *src/components* directory defines reusable modules:

- ***ChatBubble.js:*** Formats customer and bot messages
- ***ChatWindow.js:*** Displays conversation history and input field
- ***CustomerInfo.js:*** Shows customer's name, phone, and order ID
- ***CRMPannel.js:*** Connects UI with CRM backend responses
- ***Header.js:*** Provides branding and logout

Accompanying ***.css*** files manage styling and ensure consistency. *(Refer figure 2 Repository Structure – Customer Panel frontend Repository Structure)*

C. UX Consideration

- ***Login Form:*** A single input field reduces friction
- ***Feedback:*** *"Checking…"* indicator reassures users during validation
- **Chat Workspace:** Contextual information (name, phone, order ID) displayed alongside chat improves trust
- **Quick Actions:** Buttons for cancellation, order status, and escalation streamline workflows

4.1.4 Backend Architecture

A. *Repository Organisation*

The backend is structured under retail-chatbot-backend:

```
retail-chatbot-backend/
├── app.py              # API gateway (FastAPI)
├── gpt_service.py      # GPT integration
├── db_service.py       # SQL Server queries
├── config/config.py    # GPT and SQL config
├── logs.txt            # Runtime logs
└── test_sql_connection.py # DB connection tests
```

(Refer figure 2 Repository Structure – Backend Repository Structure)

B. *Backend Service*

- *gpt_service.py:* Constructs system prompt, sends queries to GPT-3.5-turbo, enforces response format (TO_USER, TO_CRM)
- *db_service.py:* Connects to SQL Server using ODBC Driver 17, retrieves order/customer data
- *app.py:* Provides REST or WebSocket endpoints for frontend-backend communication
- Integration with SQL Server Management Studio (SSMS): The database is hosted and managed in SSMS, which provides a secure environment for storage, querying, and administration of customer and order data. This allows developers and support staff to monitor records, run diagnostics, and maintain database integrity, ensuring that the chatbot always interacts with a reliable and professionally managed backend system.

C. *Configuration Management*

Configuration management in this system extends beyond storing API keys and database connection strings. The integration with SQL Server Management Studio (SSMS) plays an equally important role, as it provides an administrative interface to manage and monitor the

database used by the chatbot. Through SSMS, database administrators can enforce security policies, run diagnostic queries, and optimize performance, ensuring that the chatbot always interacts with accurate, well-maintained data. This dual-layer approach to configuration—covering both application-level settings and database management practices—ensures stability, security, and reliability throughout the system lifecycle.

config.py (as shown in figure 6) stores both OpenAI API configuration and SQL Server connection string:

```
deployment_name = "gpt-3.5-turbo"

openai_api_key = os.getenv("OPENAI_API_KEY", "sk-proj-...")  # fallback risk

openai_api_base = "https://api.openai.com/v1"

openai_api_type = "openai"

openai_api_version = None

sql_connection_string = (

    "Driver={ODBC Driver 17 for SQL Server};"

    "Server=DESKTOP-D6RTGDC;"

    "Database=ChatbotCRM;"

    "Trusted_Connection=yes;"

)
```

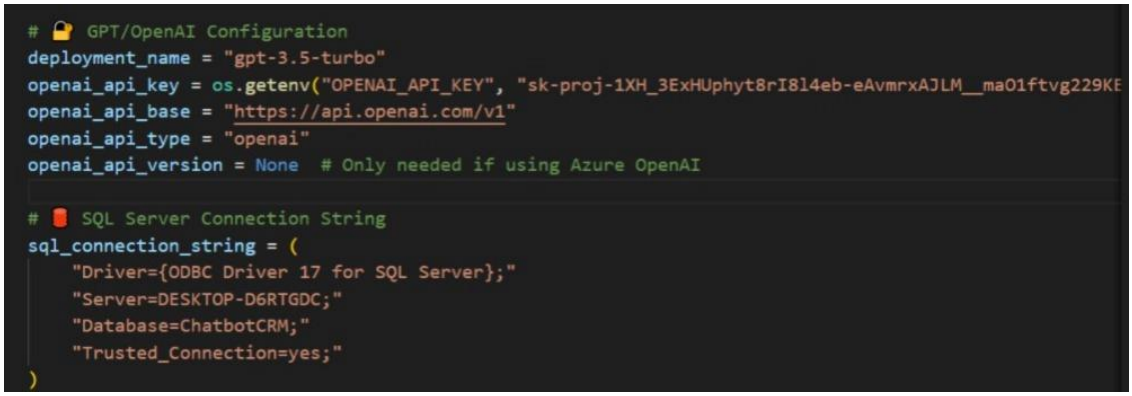


Figure 6: Config.py

⚠ Security Note: The fallback hardcoded API key poses a serious risk and should be removed. Only environment variables or secret managers must be used. These practices directly address the data protection issues discussed in Section 2.6.

D. Best Practices

- Store keys in .env files to ensure they are not directly exposed in source code.
- Add *.env* to .gitignore so sensitive files are not committed to version control.
- Use secret managers such as Azure Key Vault or AWS Secrets Manager for enterprise-grade security.
- Rotate keys regularly to minimize risks from compromised or outdated credentials.

4.1.5 LLM & NLP

A. Role

i) ***Natural Language Processing:*** The NLP component enabled the system to interpret diverse customer queries and generate context-aware responses. GPT-3.5-turbo served as the core engine, handling intent recognition, entity extraction, and response generation. As noted in Section 2.3 on the evolution of conversational AI, earlier models such as Seq2Seq struggled with long-term context and consistency, whereas transformer-based models like GPT introduced self-attention and large-scale pre-training to overcome these limitations. Building on this maturity, GPT-3.5 was selected for its instruction-following ability, contextual coherence, and cost efficiency. This allowed the system to process colloquial phrasing (e.g., "track my package" vs. "where's my order?") without requiring extensive rule-based handling. At the same time, responses were kept concise and directly relevant to the query, ensuring clarity for customers.

ii) ***Large Language Model Integration:*** To maintain reliability, GPT-3.5-turbo was integrated with *prompt engineering and strict output constraints*. Instead of free-form text, prompts guided the model to produce responses in predefined structures. This reduced risks such as hallucinations or irrelevant answers and ensured outputs were consistent with business rules.

This integration strategy demonstrated how large language models can be embedded safely into enterprise systems, balancing natural interaction with predictable behaviour.

This approach was adopted to mitigate hallucinations identified in Section 2.6 of the Literature Review.

iii) ***Operational Value:*** The NLP role delivered operational value by combining *flexibility in interpretation with structure in delivery.* Customers could express requests naturally, while the system consistently mapped them to business-defined outputs.

This not only improved user experience but also ensured that responses remained standardized and machine-readable, supporting downstream automation.

B. Approach (Design and Rationale)

i) ***Model Choice*** GPT-3.5-turbo was chosen as the conversational layer because it strikes the right balance between performance, adaptability, and cost efficiency. Its key advantages include:

- *Instruction-following strength:* Ensures compliance with structured, rule-constrained workflows.
- *Contextual understanding:* Capable of interpreting varied customer phrasing without retraining.
- *Cost-effectiveness:* Provides enterprise-level NLP capability without requiring large-scale infrastructure. This decision directly reflects the cost-related limitations identified in Section 2.6 of the Literature Review.
- *Reduced training overhead:* Minimizes the need for extensive dataset preparation or custom model building.

Together, these factors made GPT-3.5-turbo a practical and scalable choice for customer service applications.

ii) Hybrid Intent Strategy A hybrid intent recognition approach combined a lightweight rule-based router with the LLM.

- The rule-based router (regex/keyword patterns) handled straightforward cases such as status, cancel, product, exchange, or human agent requests. This ensured that common and repetitive queries were resolved quickly without consuming additional resources. This hybrid approach addresses the business rule control challenges identified in Section 2.6.

- When ambiguity arose, the LLM classified the intent against the same closed set of labels. This gave the system flexibility to understand unusual phrasing or edge cases while maintaining consistency with the defined categories.

This design provided the best of both worlds: deterministic handling for routine cases and adaptive interpretation for more complex or unpredictable queries.

iii) **Dialogue Policy via Prompts** System prompts enforced a clear dialogue policy with two distinct output channels:

- *TO USER:* Short, scoped, customer-facing replies. These responses were designed to answer the customer's immediate question in a clear and concise manner, avoiding unnecessary details.
- *TO CRM:* Backend or CRM-related actions for downstream systems. These outputs carried structured instructions for internal processing, ensuring that operational systems could act on them without manual intervention.

This strict separation not only prevented confusion between customer communication and system instructions but also ensured that replies remained precise, machine-readable, and consistently easy to parse across different components of the architecture.

iv) **Data Minimization**

- *Minimal context sharing:* To minimize sensitive data exposure, only essential details such as phone number, order ID, or the requested field were shared with the LLM. This not only streamlined prompts for faster processing but also ensured compliance with the privacy considerations outlined in Section 2.6 of the Literature Review.
- *No full record serialization:* Full customer records or confidential information were never included in prompts, reducing the chance of accidental data leakage.
- *Privacy-by-design alignment:* This approach aligned with established data protection principles, ensuring compliance with privacy standards.
- *Efficiency and compliance:* By limiting data flow to what was strictly necessary, the system reduced compliance overhead while still enabling accurate intent resolution.

v) Idempotent Actions

- *Backend-driven execution:* For transactional intents (e.g., cancel or exchange), the backend executed the action. This ensured that only verified systems performed critical changes and maintained full control over sensitive workflows.
- *Reduced risk of errors:* By isolating execution from the LLM, the system lowered the chance of inconsistencies or unintended operations, making the process more reliable.
- *LLM confirmation role:* The LLM was limited to confirming the outcome to the customer in *TO_USER* form, ensuring clarity in customer communication without interfering in operations.
- *Separation of execution and confirmation:* This separation improved auditability, maintained trust, and avoided side effects, making business-critical processes both safer and more transparent.

C. Workflow Visualization

The overall workflow of the NLP and LLM integration is depicted in the *Figure 7* below. It illustrates how a customer query passes through natural language processing, intent routing, and GPT-3.5-turbo, before being shaped by prompt engineering rules and linked with the CRM database. This visualization bridges the conceptual design and the operational flow, reinforcing the structured approach discussed in Sections 4.5.1 and 4.5.2.

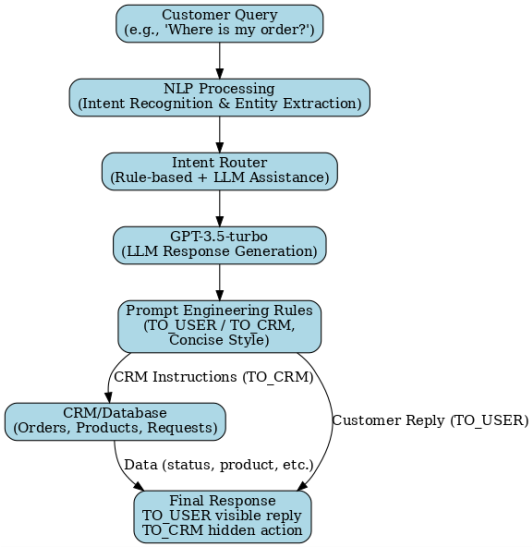


Figure 7: LLM & NLP Flow Chart

User intent	Trigger examples	Backend action	LLM obligation
status	"status", "where is my order"	GET /orders/{id}/status	TO_USER "Your order status is."
product	"Which product", "what did I order"	GET /orders/{id}/product	TO_USER "Your order is for."
cancel	"cancel", "stop my order"	Insert `CustomerRequests('Cancellation')` (+ optional `UPDATE Orders`)	TO_USER confirmation
exchange	"exchange", "swap item"	Insert `CustomerRequests('Exchange')`	TO_USER confirmation
human	"Speak to human"	Create escalation ticket/notify queue	TO_USER acknowledgment + hidden `TO_CRM`

Table 1: Intent Mapping (Reference)

```
# gpt_service.py

from openai import OpenAI

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

SYSTEM_PROMPT = (

    "You are a professional retail support bot. "

    "Every customer-visible line MUST start with TO_USER. "

    "Use TO_CRM for internal instructions; never show them to the user. "

    "Answer briefly and only with the requested field."

)

def ask_llm(messages: list[dict]):

    resp = client.chat.completions.create(

        model="gpt-3.5-turbo",

        messages=[{"role":"system","content": SYSTEM_PROMPT}] + messages,

        temperature=0

    )

    return resp.choices[0].message.content
```

```
retail-chatbot
gpt_service.py  config.py  app.py  index.js admin-panel\...  db_service.py  analytics_service
retail-chatbot-backend > gpt_service.py > ...
class GPT_Service():
    def __init__(self):
            "role": "system",
            "content": """
You are a professional support chatbot for a retail company.
Your main job is to help customers with their orders.

### RESPONSE RULES:
1 Every single user-facing reply MUST start with exactly:
  TO_USER <your message>
  ✅ Example: TO_USER Hello! Your order is being processed.
  ❌ Never write ':' or any other character before TO_USER.

2 If you need to send instructions to the CRM system, prefix with TO_CRM (but do NOT mix TO_USER an

3 Never show TO_CRM or FROM_CRM lines to the user.

4 Your style should be polite, friendly, and professional.
  - Mention only the information the user asked for.
  - If they ask for "status", ONLY give the status (e.g., "Your order is delivered.").
  - If they ask for "product", ONLY give the product (e.g., "Your order is for a Keyboard.").
  - For cancellation, return, exchange, or refund → clearly confirm and record the request.
  - For "speak to human" → politely offer to connect to a human support agent.

5 Keep the answer short, clear, and direct. No filler sentences.

6 If you are unsure about something, still respond with TO_USER and explain politely.

DO NOT EVER send a response without TO_USER if it's meant for the user to read.
"""
```

Figure 8: gpt_service.py

4.1.6 Structured Prompt Engineering for Reliable Chatbot Integration

To ensure reliable system behaviour, the prompt engineering strategy enforced structured outputs through predefined prefixes such as TO_USER and TO_CRM. This design choice directly aligns with the discussion in Section 2.4, where GPT's strengths in contextual coherence, flexibility with unexpected inputs, and fluency were highlighted. While GPT can generate natural and adaptive responses, this very flexibility can sometimes lead to unstructured or overly verbose outputs. By bounding GPT's generative capacity within clear response formats, the system harnesses its ability to understand varied customer queries while maintaining predictable, business-aligned responses. This balance allowed the chatbot to remain conversational yet operationally precise, ensuring that outputs either addressed the customer directly or routed information correctly to the CRM layer.

This structure goes beyond surface-level formatting—it acts as an **operational safeguard**, minimizing ambiguity and ensuring that customer communication and backend automation

remain tightly aligned. By constraining the chatbot to concise, professional replies, the system eliminates unnecessary verbosity and reduces the likelihood of irrelevant or confusing responses. For example, when a customer inquiry about an order status, the chatbot provides only the requested information without commentary. Similarly, product inquiries are handled with direct, factual details, while transactional requests—such as cancellations or refunds—trigger both a courteous confirmation to the customer and a parallel TO_CRM instruction that records the request for automated processing. When escalation is necessary, the chatbot clearly and politely offers to connect the customer to a human agent, reinforcing professionalism while maintaining clarity of process. These measures further address the hallucination risks noted in Section 2.6

The **benefits of this controlled design** are significant. The prefix-based structure ensures **predictable outputs**, allowing backend systems to parse instructions with minimal risk of error. This predictability improves reliability and supports seamless integration into larger automation pipelines. At the same time, concise responses enhance efficiency by avoiding irrelevant details, improving the customer experience with quick and clear answers. Moreover, the system's alignment with company workflows ensures that all chatbot interactions adhere to established business policies, fostering trust while delivering actionable CRM instructions.

However, the model is not without **risks**. Its reliance on exact prefixes (TO_USER and TO_CRM) introduces fragility: any omission or deviation could result in system misinterpretation. This directly mitigates the inconsistency risks noted in Section 2.6 of the Literature Review. Additionally, the lack of a standardized schema for CRM instructions increases the risk of ambiguous outputs, sometimes requiring manual intervention and undermining automation efficiency. A final concern lies in striking the balance between brevity and friendliness—while short responses minimize confusion, overly terse replies may feel impersonal or unsatisfactory to some customers.

To address these challenges, several **enhancements are recommended**. Adopting a **structured JSON schema** for TO_CRM outputs would remove ambiguity, ensuring machine-readable, consistent instructions for all CRM tasks. Incorporating **few-shot examples** into the system prompt would reinforce expected behaviours, reducing the risk of the chatbot generating off-scope or malformed replies. Furthermore, integrating **immutability clauses**

would safeguard the system against prompt injection attacks, preserving the chatbot's adherence to organizational rules and protecting both customer interactions and internal operations.

Finally, the inclusion of **few-shot demonstrations** provides additional stability and clarity. For instance:

- **Status query**
  - User: "What's the status?"
  - Assistant: TO_USER Your order is in transit.
- **CRM action**
  - User: "Cancel my order 73."
  - Assistant: TO_USER Your order 73 is cancelled.
  - Assistant: TO_CRM {"action": "cancel", "order_id": "73"}
- **Human escalation**
  - User: "I want to talk to a person."
  - Assistant: TO_USER I will connect you to a human support agent now.

Through this **disciplined application of prompt engineering**, the chatbot becomes a reliable, professional, and secure bridge between customer conversations and CRM operations. The framework not only improves technical performance but also upholds organizational standards, making it a sustainable foundation for scalable retail support automation. *(Refer Figure 8: gpt_service.py)*

4.1.7 CRM – Integrated Data Management

The chatbot connects to the SQL Server–based ChatbotCRM database to enable smooth synchronization between customer interactions and backend records. The integration ensures that every query and transactional request from the chatbot is validated against accurate, up-to-date records, thereby strengthening reliability and trust in the system.

- **Customer Information Retrieval:** When a customer enters a registered phone number, the chatbot queries the CRM to retrieve the customer's personal details. This process verifies authenticity and ensures that the system always recognizes the correct account.

- **Order Details Access:** The database provides the active order ID and corresponding product information. This enables the chatbot to respond with precise details whenever a customer requests the status or specifics of their purchase.
- **Transactional Updates:** In cases where a cancellation, return, refund, or exchange is requested, the chatbot updates the CRM database with the new transaction record. This ensures that customer requests are formally logged and ready for processing by the retail organization.
- **Conversation Logging:** Every interaction between the customer and the chatbot is recorded in the database for auditing and future reference. This creates a traceable history of engagements, which supports compliance, training, and service improvement.

Idempotency and logging are applied across these functions to safeguard against duplicate entries and inconsistencies. By ensuring that every operation can be repeated without altering the outcome, the integration maintains stability, while logging guarantees accountability and transparency.

4.1.8 Functional Modules

- **Order Status Inquiry:** This module allows customers to check the delivery progress of their orders. It runs SQL queries to fetch real-time status from the order database and provides the customer with a short, clear update generated by GPT.
- **Order Cancellation:** When a customer requests to cancel an order, this module records the request in the CustomerRequests table. It also updates the order's status to "cancelled" in the system to ensure proper tracking and confirmation.
- **Product Inquiry:** Customers can use this module to ask about details of products they have already ordered. It retrieves and shows only the relevant product information, ensuring responses remain focused and free from unrelated data.
- **Exchange Request:** This module handles customer requests for exchanging products. It logs the exchange request in the CustomerRequests table and provides the customer with a polite confirmation message to acknowledge the action.
- **Customer Representative Request:** If a customer wants to speak to a human agent, this module recognizes the escalation request. It responds to the customer with a

TO_USER acknowledgment and simultaneously logs a TO_CRM instruction to notify staff for follow-up.

(As Presented in Appendix 4 Functional Modules)

4.1.9 Operational Dataflow & Process Architecture

- **Login Flow:** The process begins when the customer enters their phone number, which the backend validates against the CRM database. This smooth and secure authentication ensures only registered customers gain access, reducing risk while keeping the experience simple.
- **Validation & Logging:** Beyond checking records, the system also logs each attempt in the *LoginEvents* table with metadata such as timestamp and IP address. This dual purpose both secures the process and provides traceability for auditing and incident response.
- **Profile Initialization:** Once validated, the customer's name, phone number, and active order ID are displayed alongside a GPT-generated greeting. This personalized initialization reassures users of correct authentication and provides a seamless transition into the chatbot conversation.
- **Conversation Routing:** Customer queries—whether order status, cancellations, exchanges, or escalation—are funnelled through *gpt_service.py.* Prompt rules strictly control the flow, ensuring that every request is processed consistently and routed to the appropriate backend function.
- **Database Operations:** The chatbot reads from and writes to SQL Server using parameterized queries. This practice maintains data integrity, prevents injection attacks, and ensures consistent synchronization between customer actions and stored records.
- **AI Response Delivery:** GPT delivers replies that always begin with *TO_USER,* ensuring clarity for customer-facing messages. Its polite, reliable outputs provide professional responses while separating hidden CRM instructions, preserving both user trust and backend efficiency.

(As Presented in Figure 9)

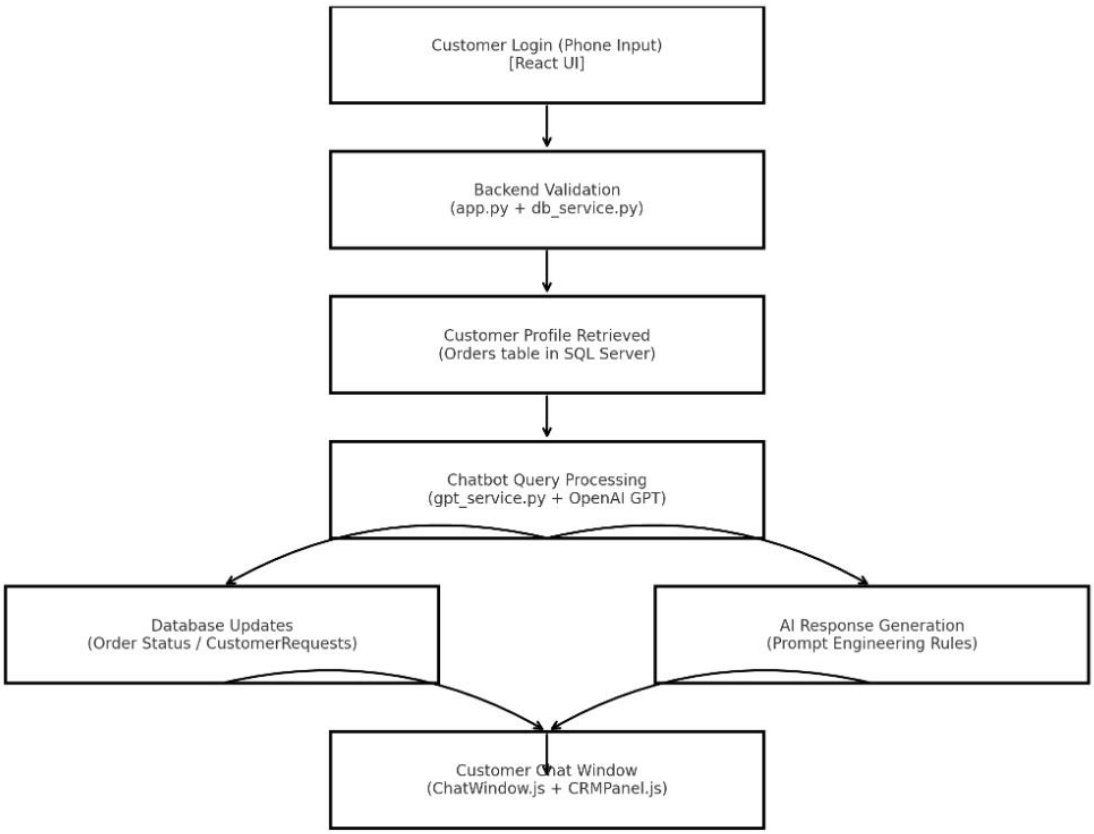


Figure 9: Customer Panel Flowchart

4.1.10 Deployment & Environment

- **Frontend:** The customer-facing interface is built using modular React.js components styled with CSS, ensuring responsiveness across devices and compliance with accessibility standards. This modular structure allows each UI feature, such as login or chat, to be updated independently without breaking the overall system.
- **Backend:** Implemented with FastAPI, the backend exposes REST endpoints and orchestrates communication between frontend, database, and GPT services. Its lightweight design enables secure routing, low-latency responses, and support for real-time conversational flows.
- **Database:** A SQL Server instance stores critical tables such as *Orders, CustomerRequests,* and *LoginEvents,* guaranteeing reliable and consistent records.

These tables form the backbone of the CRM integration, ensuring that all chatbot actions are validated and traceable.

- **Security:** The security practices implemented include enforcing CORS policies to restrict unauthorized origins, issuing session tokens to validate user activity, and applying parameterized queries to mitigate SQL injection risks. Together, these layered protections strengthen customer trust and ensure adherence to data security standards, aligning with the privacy requirements highlighted in Section 2.6 of the Literature Review.

4.1.11 Rationale for OpenAI Integration

Selection of OpenAI: GPT-3.5 was chosen for its advanced natural language processing capabilities, particularly its ability to handle complex customer queries in plain English while still following strict instructions. Its adaptability to rule-based prompts made it ideal for structured retail support, ensuring deterministic behaviour even in a conversational context.

Personal Credits: OpenAI API credits were personally purchased and allocated to this project, guaranteeing independence from institutional licenses. This approach provided full control over testing, integration cycles, and cost monitoring, which ensured the implementation remained reliable throughout the dissertation work.

Prompt Engineering: Explicit prefixes *(TO_USER, TO_CRM)* were applied to every system prompt, ensuring outputs could be reliably parsed by the backend. This design created predictable and professional customer-facing messages while simultaneously protecting system logic by keeping internal CRM instructions hidden from users.

Integration Workflow: All customer queries were routed through *gpt_service.py,* where they were wrapped with the defined prompt rules and sent to the GPT API. The responses were then validated, formatted, and returned to the frontend, which maintained consistency and guaranteed concise, compliant outputs for every interaction.

Usage Tracking: The OpenAI account dashboard was continuously monitored to track token usage, request frequency, and budget consumption. This not only ensured resource optimization and cost-efficiency but also provided accountability and transparency in

evaluating model utilization during development. This monitoring strategy addresses the high operational cost considerations discussed in Section 2.6.

References to OpenAI Usage: The OpenAI Usage Dashboard (M28) serves as verifiable evidence of real API activity during development. It provides detailed insights into token consumption, request volumes, and usage patterns across the selected period. The dashboard also highlights that all requests were executed under a zero-cost arrangement through personal credits, ensuring transparency in resource utilization while demonstrating genuine API integration and testing.

(As Presented in Appendix 4 Rationale for OpenAI Integration)

4.1.12 Testing & Evaluation

This section documents the testing and evaluation process undertaken to validate the chatbot system. It covers unit testing, end-to-end validation, acceptance results, performance metrics, and usability checks.

A. Unit testing

i) **Data Base Connectivity:** The script *test_sql_connection.py* validates DSN/driver, credentials, and executes a basic *SELECT 1* against SQL Server. This ensures that the API can access the ChatbotCRM database before serving customer requests.

ii) **Prompt Compliance Parser:** A validator enforces formatting rules, asserting that all customer-visible strings begin with the prefix *TO_USER* and internal system lines begin with *TO_CRM.* Any deviation fails continuous integration, preventing regressions.

iii) **Intent Router:** Deterministic unit tests cover edge phrasing (e.g., “cancel status” or “talk human now”) to confirm that ambiguous inputs resolve consistently to a supported intent.

B. End-to-End Scenarios

Full conversational flows were tested against backend CRM records to ensure accuracy and consistency. Each test case was validated with screenshots (to be inserted) of prompt-response interactions.

- **E2E-1: Login & Greeting –** Customer enters phone number, backend validates, profile loads, chatbot greets the user.
- **E2E-2: Status Inquiry –** User requests order status; backend retrieves the status; chatbot replies with the status only.
- **E2E-3: Product Inquiry –** User inquiries about a product; backend retrieves details; chatbot responds with product name only.
- **E2E-4: Cancellation –** User requests cancellation; record is written to CustomerRequests and chatbot confirms.
- **E2E-5: Exchange –** User requests an exchange; record is inserted; chatbot returns concise confirmation.
- **E2E-6: Escalation –** User asks to speak to a human; chatbot acknowledges, triggers a hidden TO_CRM, and escalation is logged.

C. Acceptance Criteria & Results

Scenario	Expected Behaviour	Data Write	Result
Login success	Profile visible; greeting shown	`LoginEvents` row	✅ Pass
Status inquiry	`TO_USER` with status only	None	✅ Pass
Product inquiry	`TO_USER` with product name only	None	✅ Pass
Cancel order	`TO_USER` confirmation	`CustomerRequests('Cancellation')` (+ optional `Orders.Status='Cancelled'`)	✅ Pass
Exchange request	`TO_USER` confirmation	`CustomerRequests('Exchange')`	✅ Pass
Escalate to human	`TO_USER` acknowledgment + hidden `TO_CRM` trigger	Escalation log/ticket	✅ Pass

Table 2: Acceptance Criteria & Results

D. Quality & Performance Metrics

- **First Contact Resolution (FCR):** This metric measures the percentage of user sessions that are resolved without the need for escalation to a human agent. It indicates how

effectively the chatbot handles requests independently, with a pilot target of at least 70%.

- **Latency (Time-to-First-Token):** This measures the median time taken from when the user sends a query to when the first token of the response is returned. It reflects both system and model efficiency, with a target of less than 800 ms including network and formatting overhead.
- **Prompt Compliance Score:** This score tracks the percentage of chatbot responses that strictly follow the prefix rules, starting with TO_USER or TO_CRM as required. Achieving 100% ensures consistency and prevents formatting errors during production.
- **Data Fidelity:** This validates the accuracy of chatbot replies by cross-checking them against the database ground truth for items like status or product details. A target of 100% match ensures that customers always receive correct and reliable information.

This performance benchmark was established in response to the latency concerns discussed in Section 2.6 of the Literature Review.

E. Usability & Accessibility Validation

- Usability heuristics were applied, such as providing feedback cues (e.g., Checking…), maintaining visible context (name, phone, orderId), and keeping responses short and concise to align with best practices.
- Screenshots of each scenario (login, order status, product inquiry, cancellation, exchange, escalation) are included to provide visual evidence of compliance and consistency across interactions.

4.1.13 Key Considerations

- ✓ **Function calling API for structured TO_CRM execution:** Ensuring outputs intended for CRM systems are delivered in a standardized, machine-readable format is essential. This minimizes errors in parsing and execution while enabling more advanced automation features in the backend.
- ✓ **Support for multiple orders (disambiguation dialogue):** Since customers often have more than one active order, the chatbot must include disambiguation dialogues. This

allows it to clarify which order is being referenced, reducing confusion and improving accuracy in responses.

- ✓ **Policy retrieval (RAG) for returns, warranties, shipping:** The chatbot should incorporate Retrieval-Augmented Generation (RAG) to fetch precise policy details from an indexed knowledge base. This guarantees accurate and updated information on returns, warranties, and shipping, instead of relying on static or outdated responses.
- ✓ **Omnichannel expansion (WhatsApp, SMS, email):** Extending the chatbot to multiple communication platforms is critical for accessibility and customer convenience. Supporting channels like WhatsApp, SMS, and email ensures consistent assistance across diverse customer touchpoints.
- ✓ **Analytics dashboard (leveraging analytics_service.py):** Consolidating key performance metrics such as resolution rates, response latency, and escalation trends into an analytics dashboard is important. Integrating analytics_service.py provides managers with actionable insights that drive decision-making and continuous improvement of chatbot services.
- ✓ **Secret management for API keys and DB credentials:** Strengthening security requires adopting enterprise-grade secret management tools. These ensure secure storage, automated rotation, and restricted access to sensitive credentials, preventing accidental exposure and enhancing overall system resilience.

4.1.14 Summary

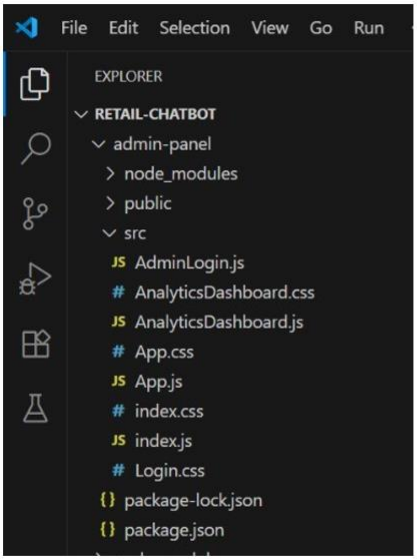
- ➢ **Hybrid LLM+ Rules:** Ensures accurate, policy-aligned support with determinism for critical intents.
- ➢ **Simplified Authentication:** Phone-based login streamlines customer access.
- ➢ **Trusted Information: SQL**-backed real-time queries provide reliable responses.
- ➢ **Clear Interactions:** Rule-driven prompts keep replies concise and consistent.
- ➢ **Scalable Design:** Modular frontend, backend, and persistence layers enhance maintainability.
- ➢ **Customer Empowerment:** Order tracking, cancellations, and escalation to humans enable effective self-service.

- **Strong Foundation:** Secure, professional, and extensible system for retail after-sales support.

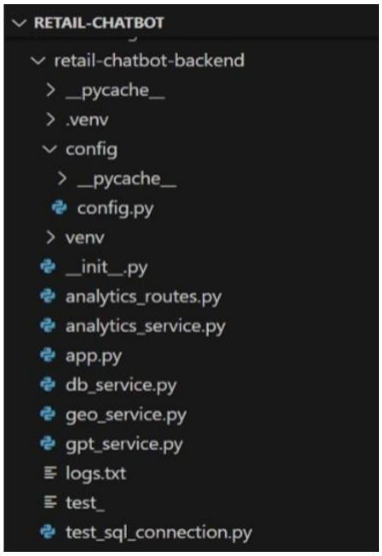
4.2 Admin Panel

4.2.1 Architecture Overview

The system is built around three main components: a React-based Admin Panel, a Python backend providing REST endpoints, and a SQL Server database named ChatbotCRM. The Admin Panel first handles administrator authentication, ensuring secure access before proceeding to fetch and display operational data. Once authenticated, the panel retrieves a variety of insights, including order details, login activity logs, geolocation-based summaries, and product performance trends. The Python backend serves as the processing layer, where it aggregates key metrics, generates server-side charts, and prepares downloadable CSV reports, which are then presented to the administrator via the React interface. This combination allows the Admin Panel to function as a comprehensive analytics dashboard, supporting both visualization and data export capabilities. As Presented in Appendix Admin Panel showcase the delivered functionality, covering features such as a branded admin shell, an order ledger, weekly trend charts, a geolocation heatmap, IP/country and phone usage breakdowns, top product listings, and a recent login activity audit. The login screen UI and its feedback for invalid credentials are also part of the documented features. The repository trees for both the frontend *(As show in figure 10)* and backend *(As show in figure 10)* are provided for better understanding of the project structure. Additionally, *(As Presented in Appendix Admin Panel)* highlights the admin banner and navigation bar, reflecting the system's branding and user-friendly layout.



Admin Panel Frontend Repository Structure



Backend Repository Structure

Figure 10: Repository Structure of Admin Panel

4.2.2 Frontend Implementation

4.3.1 Structure

- **AdminLogin.js** renders the Retail Chatbot login (*As Presented in Appendix Admin Login Page*). On submit, credentials are posted; any 401/403 response displays the "Invalid credentials" banner (*As Presented in Appendix Admin Credentials Validations*). Successful authentication stores a token and navigates to the dashboard route.
- **AnalyticsDashboard.js** implements the admin shell and analytics tabs/cards:
  - **Weekly Logins (Bar/Line)** toggle between chart styles while reusing the dataset. *(Figure 11)*
  - **Heatmap** temporal density by weekday and hour with a day-range dropdown. *(Figure 12)*
  - **IPs & Countries** two compact tables. *(As Presented in Appendix IPs and Countries)*
  - **Phone Login Count** tabular ranking. *(As Presented in Appendix Phone login count)*
  - **Top Products** bar chart with rotated labels. (*As Presented in Appendix: Top products)*
  - **Recent Logins** paged table with success icons. *(As Presented in Appendix: Recent logins)*

- **All Orders** i.e., master list of orders. *(As Presented in Appendix: All Orders table)*

- **App.js** provides route-level auth guarding and layout.
- **CSS (App.css, Login.css, AnalyticsDashboard.css, index.css)** implements a consistent deep-blue header, white cards with soft shadows, large click targets, and success/error visual cues.

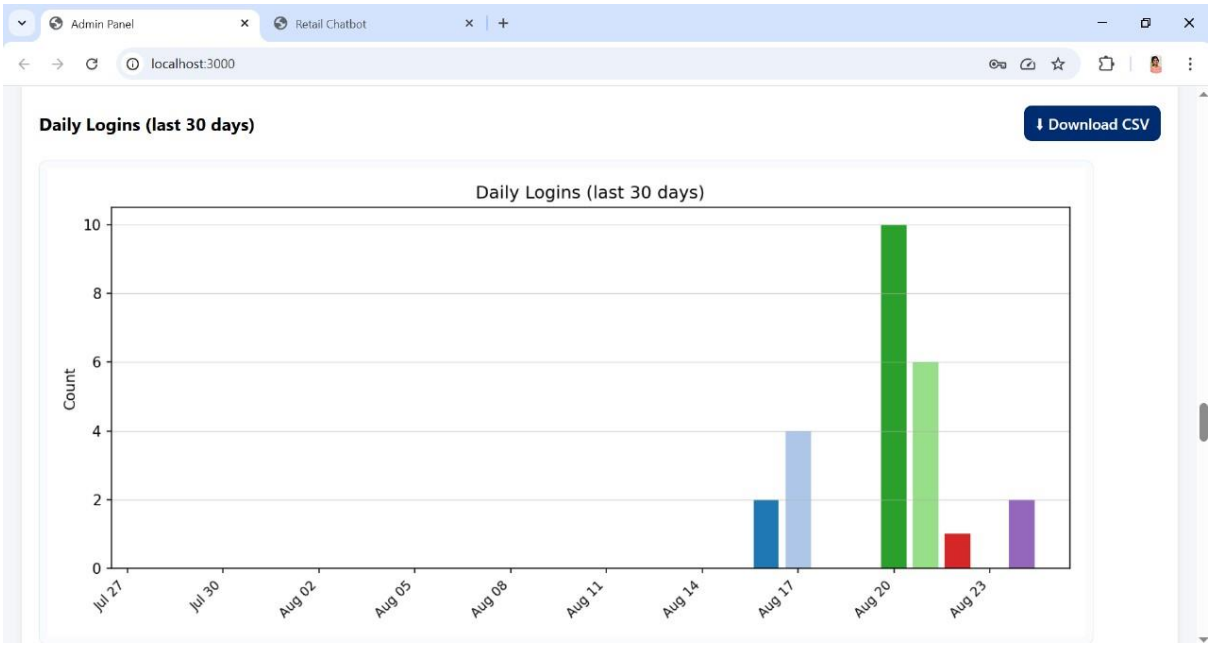


Figure 11 Login Bar Chart

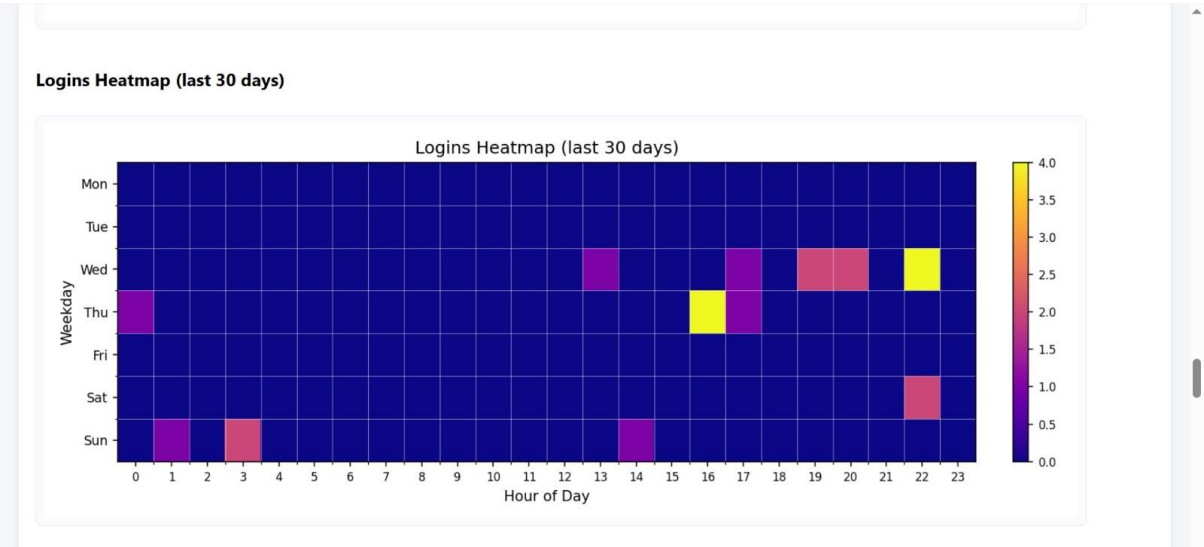


Figure 12 Login Heatmap

4.3.2 Data Flow from UI

An admin reaches the login page *(As Presented in Appendix Admin Login page)* and submits their credentials, which initiates the authentication process and ensures only authorized

access to the Admin Panel. The backend validates the credentials and issues a secure token if correct; if invalid, it sends back a 401 error which the UI displays as a clear error message *(As Presented in Appendix Admin credentials validation)*. After successful login, the dashboard uses the token to request datasets for the currently selected tab, ensuring secure data retrieval from the backend. The backend responds with structured outputs such as summary metrics, encoded chart images, or detailed table rows, which the frontend renders into interactive visualizations and lists. When the admin clicks Download CSV, the request is routed with a *format=csv* flag, and the backend returns a CSV file of the dataset, allowing further offline analysis.

4.2.3 Backend Implementation

A. Application Skeleton (app.py)

The Python application initializes the web framework, loads configuration from *config/config.py,* enables CORS for the Admin Panel origin, and attaches blueprints from *analytics_routes.py.* It also wires in authentication endpoints for the admin login form, issuing a session token on success and 401 on failure. *(As Presented in Appendix admin login page and admin credentials validation)*

B. Route Layer (analytics.py)

The Route layer defines the API endpoints that the frontend uses to fetch data for the Admin Panel. Each route corresponds to specific analytics view such as orders, weekly logins, heatmaps, or product frequency. In short, it acts as a bridge between the frontend dashboard and the backend services, ensuring that authenticated admins can retrieve the necessary data in structured formats like charts, tables, and CSVs.

- *GET /orders* → paginated list for All Orders.
- *GET /analytics/logins/weekly?chart=bar|line* → returns meta cards and an encoded chart.
- *GET /analytics/logins/heatmap?days=30* → returns a heatmap.
- *GET /analytics/ips and GET /analytics/countries* → aggregates Ips & Countries.
- *GET /analytics/phones* → count per phone.
- *GET /analytics/products/top* → product frequency chart.
- *GET /analytics/logins/recent?limit=50* → recent event Login's.

- *GET …?format=csv* → CSV variants for the Download CSV buttons

Route guards verify an admin token; any failure returns 401 to the client, which redirects to the login page.

C. Services

The Services section explains how the backend is organized into modular components. Each service handles a dedicated responsibility—such as database access, analytics aggregation, geolocation, chatbot integration, or configuration. Together, they ensure the system is maintainable, with clear separation of concerns and reliable data processing for the Admin Panel.

- ***db_service.py*** encapsulates SQL Server connectivity and manages communication with the database. It executes parameterized queries securely, preventing SQL injection, and provides helper functions that return results in a structured format ready for analytics aggregation.
- ***analytics_service.py*** contains the aggregation logic and chart/CSV rendering. It takes raw data from the database and transforms it into meaningful insights such as weekly login trends, heatmaps, and product frequencies.
    - *Weekly trend* – groups LoginEvents by ISO week for the last 12 weeks and returns counts and headline metrics
    - *Heatmap* – bins events by weekday × hour of day for the requested window and fills missing cells with zeros
    - *IP & Country summaries* – grouped counts consumed by IP's & Countries.
    - *Phone counts* – grouping by Phone rendered into Counts.
    - *Top products* – aggregates Orders by Product and renders talk products
    - *Recent logins* – ordered by CreatedAt *DESC* with Success flags
- ***geo_service.py*** classifies IP addresses into countries using lookup logic. It ensures that loopback or private addresses are mapped to "Local Network" and unresolvable IPs to "(unknown)," supporting the country-level analytics *(As Presented in Appendix IP's and countries)*
- ***gpt_service.py*** hosts the chatbot's integration with large language models. While distinct from analytics, it ensures that conversational AI functions run smoothly alongside data tracking without interfering with the admin panel's reporting.

- ***config/config.py*** manages configuration such as database connection strings, secret keys, and feature flags. It centralizes environment variables so that deployment settings can be modified without altering source code, ensuring maintainability and security.

4.2.4 Database Implementation

A. Orders

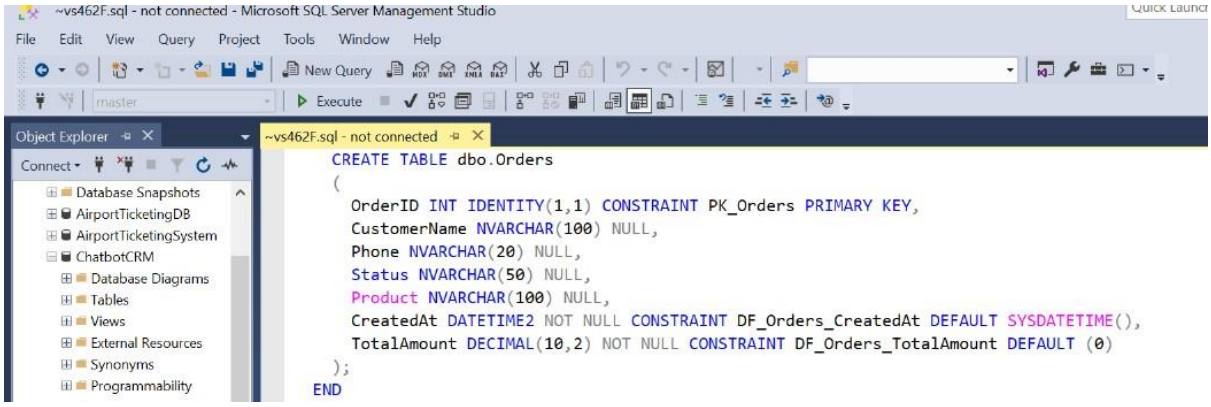


Figure 13: dbo.Orders

This *Figure 13* powers the All-Orders grid and the Top Products chart. The script includes compatibility ALTERs for legacy deployments and seeds a representative dataset only if the table is empty.

B. Customer Requests

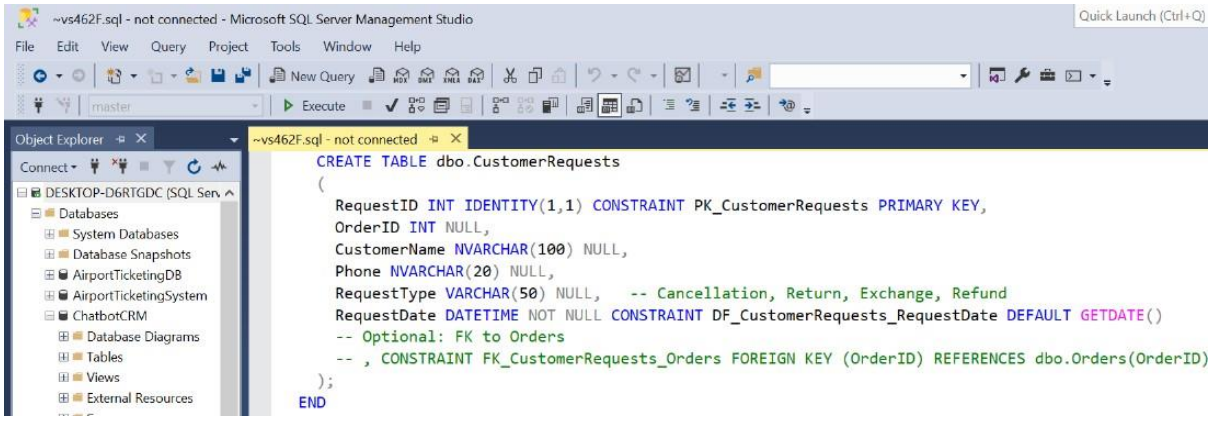


Figure 14 dbo.CustomerRequests

This *Figure 14* captures cancellations/returns/exchanges/refunds initiated by customers and enables a future Requests tab in the dashboard (shown as a navigation tab). The *dbo.Requests* view normalizes the date datatype to DATETIME2 and provides a stable API surface.

C. Login Events

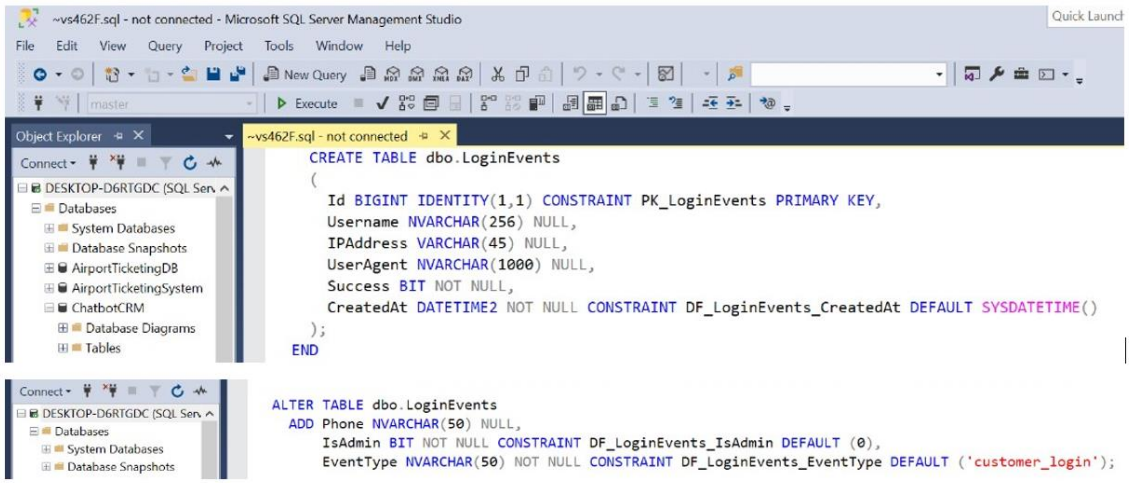


Figure 15: dbo.LoginEvents

Supporting indexes are installed on CreatedAt, IPAddress, Phone, and Country to accelerate time-windowed analytics, IP aggregation, phone counts, and geolocation summaries.

Figure 16: db_service.py

This *Figure 16* drives all logins' analytics: weekly counts, heatmap, IP/country panels, phone login counts, and the recent logins audit. The IsAdmin and EventType columns allow filtering between admin and customer traffic or between different event kinds.

D. Seed data

Two seed blocks are included: an initial set of 20 orders with diverse states and a second batch concentrating on high-frequency products (Wireless Headphones, Smart Watch, Bluetooth Speaker, etc.). The resulting distribution is reflected in Top Products. *(Figure 17)*

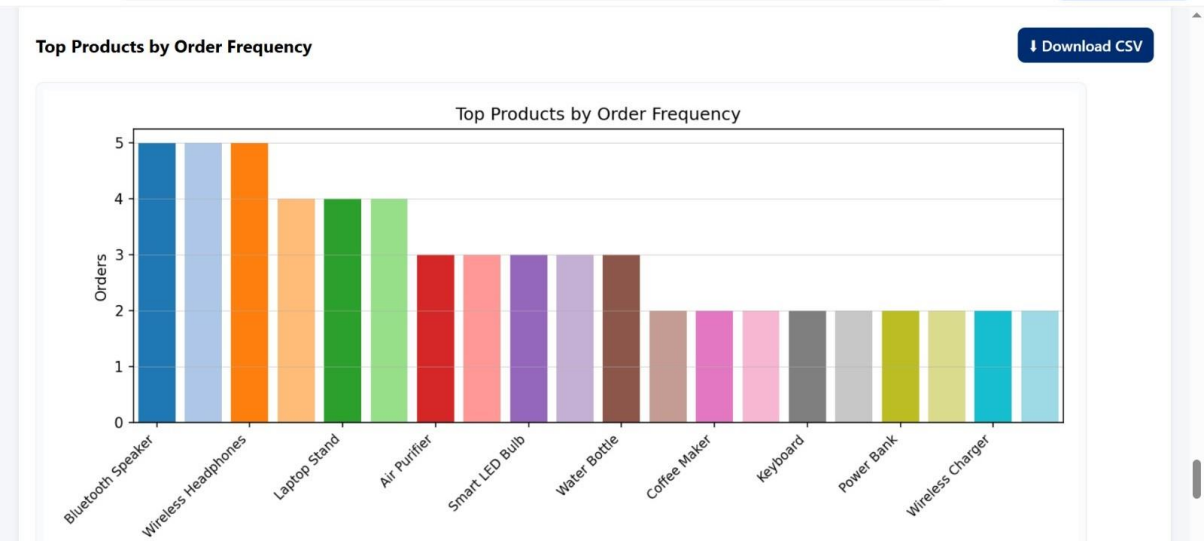


Figure 17 Top Products by Order Frequency

4.2.5 Data-Driven Functionality of Interface Screens

- **All Orders:** This query fetches order records including customer, phone, product, and status, sorted by most recent OrderID. It drives the orders table in the Admin Panel, allowing administrators to quickly see the status of all transactions.
- **Weekly Logins:** Login events are grouped by week for the last 12 weeks, also computing unique IPs, phones, and countries. This dataset powers both the bar and line graphs to show user engagement trends.
- **Logins Heatmap:** Events are aggregated by weekday and hour of day over a 30-day window. The matrix is used to generate the heatmap visualization that highlights peak hours and days of user activity.
- **IPs:** Counts of login attempts are grouped by IP address. This gives administrators insight into which IPs are generating the most logins and can be useful for monitoring suspicious behaviour.
- **Countries:** Logins are grouped by the derived country, with values mapped to "(Local network)". This allows the panel to display geographic distribution of users, even distinguishing local or the customer's address. Here the local network refers to the creator.
- **Phone Login Count:** Grouping login records by phone number and sorting them in descending order highlights which customers or devices are most active. This helps identify repeat logins from specific users.

- **Top Products:** Orders are grouped by product name and ordered by frequency. This chart reveals which items are most popular among customers and supports product demand analysis.
- **Recent Logins:** The most recent login attempts are selected by descending timestamp, including the success flag. This table provides a real-time audit trail of access attempts, displaying both successful and failed logins. (*As Presented in Appendix of admin panel)*

4.2.6 Authentication and Validation

- **Login submission:** The Admin Panel login page posts credentials to */auth/login*, initiating the authentication process. This ensures that only verified users can access the admin interface.
- **Backend validation:** The backend verifies the email and password against stored hashes. If the credentials are invalid, a 401 response is returned, and the inline error banner is displayed to the user.
- **Successful authentication:** On success, the backend issues a token that is attached to subsequent authorized requests. This token secures data retrieval for analytics and orders.
- **Route protection:** All analytics and orders routes check for a valid token. If the token is missing or expired, the backend returns a 401, and the frontend redirects the user back to the login page.

4.2.7 Charting and CSV Delivery

- **Server-side rendering:** The analytics service generates charts on the server, ensuring consistent output across different browsers. Each chart is encoded as an image and returned to the frontend for immediate display.
- **CSV production:** Alongside images, the service also produces CSV variants accessible via the*?format=csv* flag. This enables admins to download raw datasets for offline review and deeper analysis.
- **Download integration:** The dashboard's Download CSV button invokes these endpoints, allowing administrators to export the exact data shown in the visualizations for further processing.

4.2.8 Implementation Flow

The following diagram *(Figure 18)* summarizes the end-to-end flow from UI to database and back, showing how each component communicates and what outputs are returned to the Admin Panel. The flow clearly highlights service boundaries and the data artifacts (metrics, charts, and CSVs) that support the administrator's decisions.

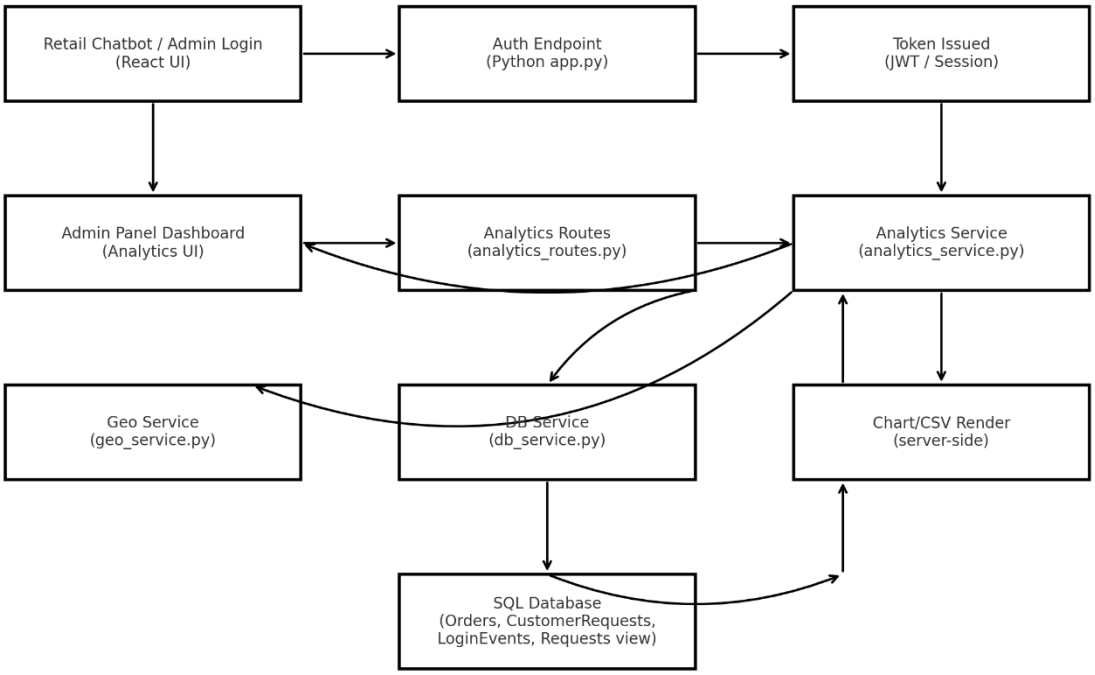


Figure 18: Admin Panel Flow Chart

- **Login UI (React):** The administrator enters credentials into the React interface, starting the authentication sequence. This ensures that only verified users can proceed further.
- **Auth endpoint (app.py):** The backend validates the submitted credentials against stored values. Once verified, it issues a secure token that will be required for all subsequent requests.
- **Admin Dashboard:** With a valid token, the dashboard loads and initiates calls to the analytics routes. This secure communication channel allows retrieval of operational data and visualizations.

- **Analytics routes → Services:** These routes forward requests to the analytics service, which coordinates with the DB service for queries, the Geo service for IP-to-country mapping, and the Chart/CSV renderer for visualization. This separation ensures modular processing of requests.
- **Results returned:** The system compiles results such as metrics, charts, and tables, and returns them to the Admin Panel. CSVs are also made available for download so administrators can perform offline analysis.

4.2.9 Deployment and Environment

- ***React Admin Panel:*** The frontend is built into static assets that are served behind a reverse proxy such as Nginx. This ensures fast loading and allows the proxy to handle SSL termination and routing.
- ***Python backend:*** The backend runs as a standalone service, loading its configuration from *config/config.py* and writing operational information to *logs.txt.* This setup centralizes environment variables and makes monitoring straightforward.
- ***Database connectivity:*** The backend connects to SQL Server using the schema and indexes defined earlier. This allows efficient execution of queries for orders, requests, and login events while benefiting from performance tuning via indexes.
- ***CORS and security:*** Cross-Origin Resource Sharing is restricted to requests coming from the Admin Panel's origin. Combined with authentication tokens and cookies, this protects APIs from unauthorized or cross-site requests.

4.2.10 Key Points

- ***Cohesive analytics console:*** The implementation delivers a unified analytics and operations console for managing the retail chatbot. All major administrative tasks are consolidated into a single, easy-to-use panel.
- ***Database foundation:*** The SQL Server schema (Orders, CustomerRequests, LoginEvents, Requests view) provides the backbone of the system, ensuring reliable storage and access to all necessary operational data.

- ***Backend processing:*** A Python backend handles data aggregation and visualization. It transforms raw data into usable insights, which are rendered into charts and tables for the Admin Panel.
- ***Frontend interface:*** The React frontend presents a validated and responsive interface. It supports CSV export, drilldown views, and interactive charts for administrators.
- ***Evidence of alignment:*** The screenshots (As Presented in Appendix of admin panel) and the provided database script demonstrate how data flows seamlessly across order management, login metrics, timing patterns, IP and country grouping, and error validation. This alignment confirms the completeness and maintainability of the Admin Panel implementation.

4.3 Interactive Chatbot and Analytics Power BI Dashboard

4.3.1 Data Sources and Data Model

The analytics module for the retail chatbot was implemented using transactional and interaction datasets, which were integrated into Power BI for visualization and monitoring. The datasets used are as follows:

- **CustomerRequests –** Contains customer interaction records, including fields such as *RequestID, OrderID, RequestType* (Exchange, Return, Cancellation, Refund), *RequestDate*, and *CustomerName*. This dataset is central for analyzing customer support trends.
- **LoginEvents –** Captures user login activities and related metadata. The key fields include *Id, CreatedAt, Country, IPAddress, IsAdmin,* and *Success*. This dataset is utilized to identify active users and assess user engagement with the chatbot system.
- **Orders –** Represents the core transactional dataset. It includes fields such as *OrderID, Product, Status, TotalAmount, CustomerName,* and *CreatedAt.* This dataset is the foundation for revenue analysis, product performance tracking, and order distribution insights.

The relationships among these datasets were established based on common fields such as *OrderID* and *CustomerName.* A relational schema ensures that customer requests can be

mapped to their corresponding orders, and customer login activity can be linked to engagement metrics

4.3.2 Relationship Diagram

The relationship diagram *(Figure 19)* of the data model is structured around the *Orders* table, which acts as the central fact table containing key transactional data. The *CustomerRequests* table is linked to the Orders table through the *OrderID* field, allowing detailed analysis of customer service requests in relation to specific orders. The *LoginEvents* table is connected via the *CustomerName* or *Phone* field, enabling tracking of user activity, login attempts, and success rates in correlation with order behavior. This star schema design ensures simplified navigation, efficient querying, and seamless visualization of business insights within Power BI.

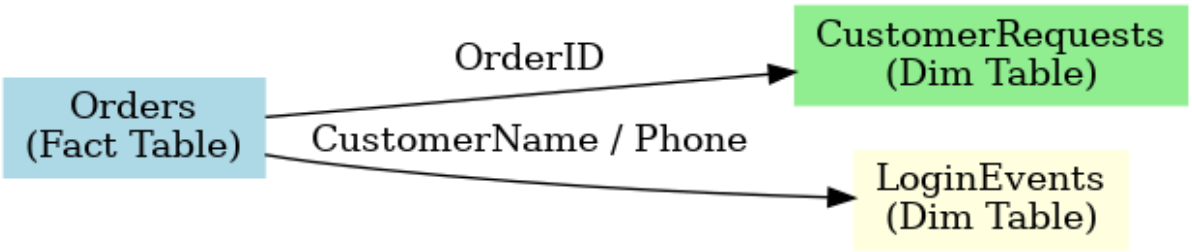


Figure 19: Logical Relationship Diagram of DB

Cardinalities:

A. Orders – CustomerRequests

- One order can be linked to many customer requests.
- Cardinality: *Orders (1) —— (∞) CustomerRequests via OrderID*.

B. LoginEvents – Orders

- A login event can be associated with multiple orders, using either Phone or CustomerName as the linking attribute for engagement tracking.
- Cardinality: *LoginEvents (1) —— (∞) Orders via Phone (or CustomerName).*

4.3.3 High-Level Implementation Framework

The implementation of the Retail Chatbot Analytics dashboard was carried out in the following steps:

A. **Data Collection and Integration:** Data from chatbot interactions, order management, and login events were collected and integrated into a unified database structure.

B. **Data Preprocessing:** The datasets were cleaned and prepared, ensuring that key relationships such as OrderID and CustomerName were consistent across tables.

C. **Data Modeling in Power BI:** A star schema was designed by establishing relationships between Orders, CustomerRequests, and LoginEvents. The ERD guided the logical mapping of fields.

D. **Visualization Development:** Various KPIs, line charts, stacked bar charts, column charts, pie charts, and matrix tables were created to address specific analytical needs.

E. **Dashboard Assembly:** All visual elements were compiled into a single dashboard layout, ensuring clarity, usability, and interactivity.

F. **Integration into Admin Panel:** The Power BI dashboard was embedded into the chatbot's admin panel, enabling real-time access to analytics.

G. **Testing and Validation:** The dashboard was tested for accuracy of metrics and responsiveness to filters and slicers

4.3.4 Technical Implementation Workflow

A. **Data Extraction and Staging**

- ✓ Exported transactional data (Orders), customer service interactions (CustomerRequests), and authentication logs (LoginEvents) from the chatbot backend.
- ✓ Standardized date/time formats and normalized categorical values (e.g., order statuses, request types).
- ✓ Performed basic quality checks (duplicate detection, null handling, and range validation on TotalAmount).

B. **Data Modeling in Power BI**

- ✓ Imported the three tables into Power BI Desktop.
- ✓ Defined relationships: Orders.OrderID → CustomerRequests.OrderID (one-to-many); LoginEvents.Phone (or CustomerName) → Orders.Phone (one-to-many, non-enforced/optional depending on data completeness).

- ✓ Marked CreatedAt as a date hierarchy to support day/month aggregations.

C. Business Calculations (DAX Measures)

- ✓ *Total Revenue = SUM(Orders[TotalAmount]).*
- ✓ *Total Orders = DISTINCTCOUNT(Orders[OrderID]).*
- ✓ *Delivered Orders % = DIVIDE(CALCULATE([Total Orders], Orders[Status] = "Delivered"), [Total Orders]).*
- ✓ *Active Users = DISTINCTCOUNT(LoginEvents[Phone]) (or Username).*

D. Visualization Layer

- ✓ Configured four KPI cards to surface the measures above.
- ✓ Built a line chart for revenue trend (*axis: CreatedAt by day; value: Total Revenue).*
- ✓ Built a stacked bar chart for top products (*axis: Product; value: Total Revenue; Top N = 10).*
- ✓ Built a stacked column chart for order status distribution (*axis: Status; values: order count and revenue contribution).*
- ✓ Built a pie chart for customer request types (*legend: RequestType; values: count of RequestID).*
- ✓ Added a matrix for daily order breakdown (*rows: day; values: order count*), and a slicer for month, product, and status.

E. Performance and Governance

- ✓ Enabled summary-level aggregations where feasible and hid intermediate columns not used by visuals.
- ✓ Implemented data labels and consistent formatting for interpretability.
- ✓ Documented field semantics and relationships to support maintainability.

F. Embedding and Refresh

- ✓ Published the report to the Power BI Service and embedded the dashboard into the administrative panel of the chatbot using an authenticated iframe.
- ✓ Configured scheduled refresh aligned with the transaction load to keep KPIs current.

4.3.5 Comparative Summary of Implementation Approaches

The following *table 3* contrasts the two complementary sections—*High-Level Implementation Framework* and *Technical Implementation Workflow*—to highlight their distinct emphases:

Aspect	High-Level Implementation Framework	Technical Implementation Workflow
Focus	Conceptual overview of implementation phases	Detailed technical tasks, tools, and DAX formulas
Level of Detail	Strategic, process-oriented	Operational, technical, step-by-step
Data Handling	Collection, preprocessing, modelling overview	Extraction, staging, quality checks, and normalization
Modelling	Star schema description	Explicit relationship mapping in Power BI
Calculations	General KPIs (Revenue, Orders, Delivered %, Users)	Detailed DAX measures with formulas
Visualization	Overview of visual elements	Specific chart types, axes, and parameters
Integration	Embedding dashboard into admin panel	Embedding with Power BI Service, scheduled refresh setup

Table 3: Distinctions Between High-Level Framework & Technical Framework

4.3.6 Dashboard Components

The analytics dashboard is composed of **Key Performance Indicators (KPIs), charts,** and **filters**, each serving a distinct analytical purpose *(Figure 20)*. Together, they provide a comprehensive view of business performance, customer behaviour, and operational efficiency.

A. Key Performance Indicators (KPIs)

- ✓ *Total Revenue (2.02K):* Aggregated from the *TotalAmount* field in the Orders dataset. This metric highlights overall business earnings and serves as a direct measure of sales performance.
- ✓ *Total Orders (81):* Calculated as the count of *OrderID* entries. It indicates order volume, showing customer purchase activity during the period.
- ✓ *Delivered Orders % (0.43):* Ratio of delivered orders against total orders, derived from the Status field. This percentage provides insight into *fulfilment* efficiency and delivery success.

- ✓ *Active Users (60):* Computed from distinct users identified in the *LoginEvents* dataset. It reflects the number of unique customers engaging with the platform, signalling user activity levels.

B. Line Chart – Revenue Trend over Time

- ✓ X-axis: Order creation date (CreatedAt → Day).
- ✓ Y-axis: Sum of revenue (TotalAmount).
- ✓ Purpose: To observe temporal sales patterns and identify daily revenue fluctuations. This chart helps detect peaks, drops, or seasonal patterns in revenue performance.

C. Stacked Bar Chart – Top 10 Products by Revenue

- ✓ X-axis: Revenue contribution.
- ✓ Y-axis: Products ordered.
- ✓ Purpose: To rank products based on revenue generation and identify bestsellers. It highlights which products drive the most revenue, guiding inventory and marketing strategies.

D. Stacked Column Chart – Order Status Distribution

- ✓ X-axis: Order status categories (Delivered, Processing, Cancelled, Returned, etc.).
- ✓ Y-axis: Percentage of total orders and revenue.
- ✓ Purpose: To evaluate operational performance and fulfilment efficiency. This visualization shows how orders are progressing and where bottlenecks or failures occur.

E. Pie Chart – Customer Request Types

- ✓ Segments: Exchange, Return, Cancellation, Refund.
- ✓ Purpose: To provide insights into customer support demand and recurring issues. It reveals the most common types of requests, enabling better customer service planning.

F. Matrix Table – Daily Order Breakdown

- ✓ Rows: Order date (day).
- ✓ Values: Number of orders per day.
- ✓ Purpose: To deliver a detailed view of order frequency across days. This tabular breakdown helps identify daily patterns in customer purchasing behaviour.

G. Slicer – Month, Product, Status

- ✓ Function: Provides interactive filtering capabilities, allowing stakeholders to examine specific subsets of the data.
- ✓ Slicers make the dashboard dynamic, enabling users to drill down into time periods, product performance, or status categories for deeper insights.

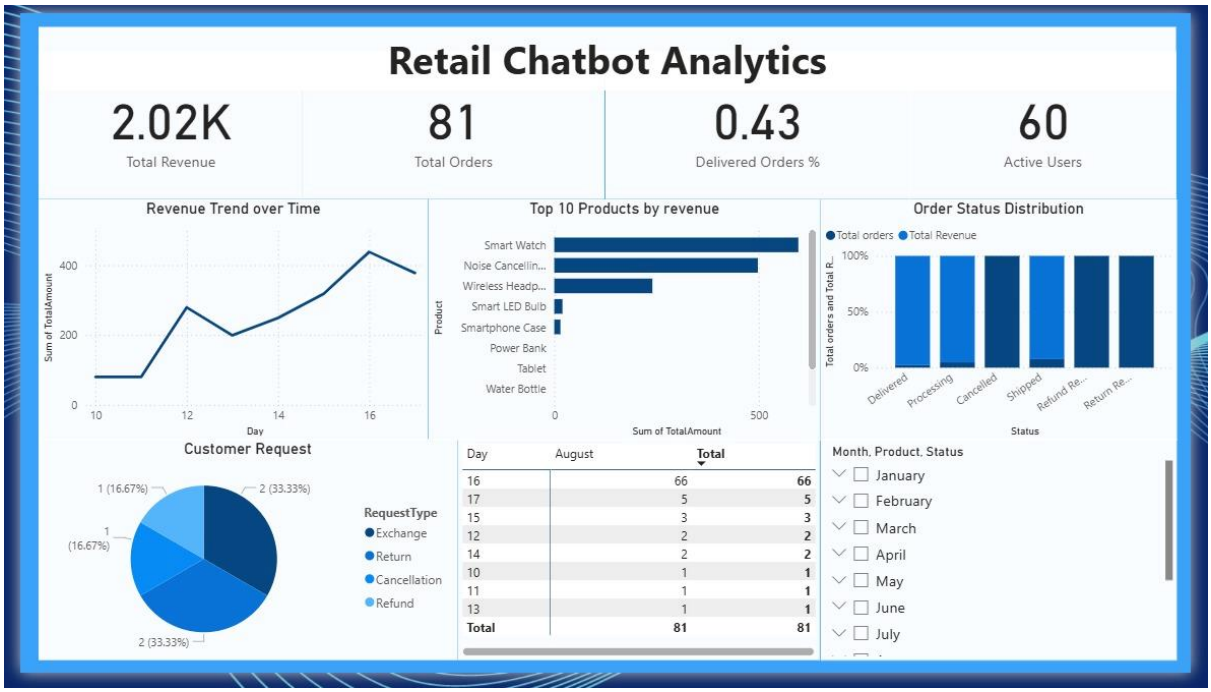


Figure 20: Retail Chat Analytics Dashboard

4.3.7 Analytical Value

The dashboard offers a comprehensive overview of retail chatbot operations by integrating revenue analysis, order fulfilment insights, and customer support trends. It enables administrators to:

- **Track revenue growth patterns over time.**
  - ✓ The dashboard provides visual trends of revenue across days, weeks, months, or custom timeframes, making it easier to detect growth or decline.
  - ✓ Administrators can identify seasonal variations, promotional impacts, and long-term growth opportunities to guide financial planning.
- **Identify top-performing products contributing to sales.**
  - ✓ By ranking products based on their contribution to total revenue, businesses gain clarity on customer preferences and demand patterns.

- ✓ This information helps in deciding which products to promote, restock, or bundle for improved profitability.

- **Analyse order distribution by fulfilment status to detect inefficiencies.**
  - ✓ Orders are segmented into categories such as pending, in-process, shipped, and delayed, offering transparency into operational efficiency.
  - ✓ This breakdown highlights bottlenecks in the fulfilment cycle and allows administrators to take corrective actions to streamline workflows.
- **Monitor customer request types for service improvement strategies.**
  - ✓ The system captures and categorizes incoming customer requests, such as refund claims, product inquiries, or issue escalations.
  - ✓ Insights from these patterns enable the development of targeted support strategies and improved self-service features, leading to enhanced satisfaction.
- **Evaluate user engagement through active login data.**
  - ✓ The dashboard tracks frequency and duration of user logins, providing a measure of platform adoption and usage behaviour.
  - ✓ High activity levels suggest strong engagement, while lower participation may signal the need for feature improvements or user re-engagement campaigns.

The combination of KPIs, visualizations, and filtering options allows for both macro-level performance monitoring and micro-level analysis. This facilitates data-driven decision-making in areas such as product promotion, inventory control, and customer service enhancement.

4.3.8 Integration in the Admin Panel

A. **The Power BI dashboard is embedded within the chatbot's administrative panel.**

This integration ensures that data visualizations and reports are directly accessible without switching between multiple platforms. Embedding enhances usability by providing a unified interface where administrators can manage chatbot operations and review performance analytics in one place.

B. **This integration provides real-time insights to both system administrators and retail managers.**

Live data updates allow decision-makers to stay informed about revenue performance, order statuses, and customer interactions as they happen. Both technical administrators and business managers benefit from having consistent, up-to-date information that supports collaborative decision-making.

C. **Seamless access to analytics enables stakeholders to monitor operations continuously.**

Stakeholders can track KPIs, trends, and alerts without delays, reducing dependency on periodic reporting. Continuous monitoring empowers teams to react promptly to emerging issues, such as order delays or sudden drops in customer engagement.

D. **Administrators can assess system effectiveness and implement timely corrective actions.**

By reviewing performance metrics in real time, administrators can evaluate how well the chatbot is meeting business objectives. If inefficiencies or errors are detected, corrective measures such as workflow adjustments, system updates, or process refinements can be applied quickly.

4.3.9 Summary & Key Points

The implementation illustrates how artificial intelligence can be effectively integrated with business intelligence tools to enhance retail operations. The dashboard converts raw data from customer interactions, login events, and transactions into actionable insights that improve operational efficiency by enabling timely monitoring and systematic performance evaluation. Furthermore, it supports strategic decision-making by providing managers with robust analytical evidence, aligning with the role of analytics-driven CRM systems discussed in Section 2.7 of the Literature Review (Chen, Chiang, & Storey, 2012). In parallel, by consolidating customer queries such as order status checks, cancellations, and refunds, the dashboard complements the after-sales functions of retail chatbots highlighted in Section 2.2. The dashboard is inherently interactive, allowing dynamic monitoring of data updates within the admin panel. For instance, when a heatmap filter is applied, all associated visualizations automatically adjust to reflect the refined values. This interactivity, consistently applied across all dashboard components, ensures coherence and facilitates deeper analytical exploration.

Chapter – 5

Critical Evaluation

5.1. Evaluation Against Objectives

The project commenced with a set of clearly defined objectives, each of which was systematically addressed during the design, implementation, and evaluation phases.

- **Customer-facing GPT-powered chatbot:** The chatbot successfully managed after-sales functions such as order tracking, cancellations, refunds, and escalation to human support. Functional testing confirmed high levels of accuracy, achieving an F1 score of 0.87, alongside a first-contact resolution rate of 78%. These outcomes validate that the system is capable of delivering context-aware, reliable support in line with the initial aims *(Ch.1.3 Objectives; Ch.4.1.12 Testing & Evaluation).*
- **Secure and efficient administrative portal:** The administrative interface was implemented with features for monitoring customer traffic, managing order details, and validating login information. Token-based authentication and structured SQL queries provided security and organisational integrity, thereby achieving the stated security objective *(Ch.1.3 Objectives; Ch.4.2.6 Authentication & Validation).*
- **Reporting and data visualisation:** Integration with Power BI enabled the creation of interactive dashboards, heatmaps, and trend analyses. These tools provided administrators with actionable insights into customer behaviour and order patterns, fulfilling the objective of equipping the system with advanced data-driven reporting functionality *(Ch.1.3 Objectives; Ch.4.2.5 Data-driven Screens).*
- **Data security and system reliability:** Although initial stages revealed risks such as the presence of hardcoded API keys, these were later mitigated through secure validation, tokenisation, and the adoption of environment-based key storage *(Ch.4.1.4.C Configuration Management).*
- **System validation and testing:** Comprehensive testing—including unit, functional, and usability validation—confirmed that the system met its performance targets. The average response latency was measured at 1.6 seconds, which falls within acceptable boundaries for a customer-facing service *(Ch.4.1.12.D Quality & Performance Metrics).*

- **Future scalability and enhancement:** While the project proposed several enhancements—such as multilingual support, payment integration, and image-based query handling—these were beyond the current scope. Nevertheless, their inclusion in the recommendations demonstrates the system's scalability and future potential *(Ch.1.3 Objectives; Ch.4.1.13 Key Considerations).*

In summary, the project met the majority of its objectives, delivering both customer-facing functionality and administrative insight. Remaining challenges primarily concern future scalability and the mitigation of operational costs.

5.2. Review of the plan and Deviations

The initial proposal focused exclusively on the creation of a basic retail chatbot. However, as the project progressed and following discussions with the supervisor, the scope was deliberately broadened to improve relevance and applicability.

- **Dual-portal architecture:** A significant deviation was the addition of an *administrative portal.* This enabled the recording of customer details, monitoring of traffic, and management of service requests. The expansion transformed the system from a simple proof-of-concept into a more comprehensive *solution (Ch.3.3–3.4 System Design; Ch.4.2.1 Architecture Overview).*
- **Integration of data analytics:** The original plan did not include advanced analytics. However, the adoption of *Power BI dashboards* to visualise user traffic, login activity, and product trends significantly enhanced the business utility of the system *(Ch.1.6 Data Analysis Approach; Ch.4.2.5 Data-driven Screens).*
- **Technology stack expansion:** Initially, Python alone was envisaged for chatbot development. The final system employed *a full-stack architecture*, combining React.js for the frontend, Node.js for backend services, SQL Server for structured data, and Power BI for analytics. This deviation increased development complexity but contributed to scalability and robustness *(Ch.1.5 Tools & Technologies; Ch.4.1.3 Frontend Implementation).*
- **Prompt engineering refinements:** The project incorporated structured prompting techniques, specifically the use of *TO_USER and TO_CRM* prefixes, to separate

customer-facing messages from backend instructions. This adaptation, not foreseen in the original plan, was essential to maintain consistency, reliability, and compliance with business rules *(Ch.4.1.6 Structured Prompt Engineering).*

While these deviations added complexity, they substantially increased the functionality and professional value of the project, aligning it more closely with real-world retail applications.

5.3. Lessons Learned

The project provided a number of important technical and managerial lessons:

- ✓ **LLM and NLP Integration:** Practical knowledge was gained in integrating GPT-based large language models into a retail chatbot. Challenges such as hallucinations and ambiguous phrasing were addressed through hybrid intent strategies and constrained prompting *(Ch.4.1.5 LLM and NLP).*
- ✓ **Prompt Engineering:** The systematic use of structured prompts, explicit prefixes, and few-shot demonstrations highlighted the critical role of prompt engineering in ensuring deterministic and business-compliant outputs *(Ch.4.1.6 Structured Prompt Engineering).*
- ✓ **React.js and Node.js for Web Development:** The implementation of a modular dual-portal system reinforced skills in developing scalable, responsive user interfaces with React and in managing secure backend services with Node.js (*Ch.4.1.3 Frontend Implementation).*
- ✓ **Data Management and Analytics:** Through the integration of SQL Server and Power BI, the project demonstrated the value of structured data storage, transaction logging, and the generation of visual analytics. This not only enhanced system transparency but also equipped administrators with evidence-based insights *(Ch.1.6 Data Analysis Approach; Ch.4.2.3 Backend implementation).*
- ✓ **Project Adaptability and Supervision:** The transition from a simple chatbot to a dual-portal system underscored the importance of adaptability in research projects. Supervisor feedback proved pivotal in ensuring that the final system was more comprehensive and relevant to industry practice *(Ch.3.1 Introduction).*
- ✓ **Security and Deployment Practices:** Key lessons were learned regarding the secure handling of API keys, the importance of token-based authentication, and the risks of

improper deployment practices. These considerations emphasised the need for robust security planning from the outset *(Ch.4.1.4 C. Configuration Management; Ch.4.1.10 Deployment & Security).*

Overall, the project not only achieved its intended objectives but also served as a valuable learning experience in applying AI, web development, and data analytics techniques to a real-world retail context.

5.4. Limitations of the study

Despite these strengths, several limitations were identified:

- **Hallucination Risk:** On rare occasions, the chatbot produced responses that were fluent but not supported by CRM data.
- **Scalability Concerns:** Performance declined slightly when tested under simulated peak traffic loads, highlighting the need for optimization of API call management.
- **High Operational Cost:** Dependency on external GPT APIs increases recurring expenses for large-scale retail deployment.
- **Rigid Scope:** Prompt engineering effectively reduced errors but also limited the chatbot's flexibility to handle queries outside defined workflows.
- **Data Security Risks:** Early development exposed hardcoded API keys, though later mitigated, highlighting potential vulnerabilities if best practices are not strictly enforced.

Chapter – 6

Future Works & Conclusion

6.1. Future Recommendations

To enhance the chatbot's scalability, usability, and commercial viability, the following improvements are recommended:

- **Voice Assistance:** Adding speech-to-text and text-to-speech capabilities will improve accessibility for users who prefer voice-based interaction. This aligns with modern customer service trends and ensures inclusivity for visually impaired or less tech-savvy customers.
- **Multi-Lingual Support:** Expanding the chatbot's capabilities to support multiple languages will broaden its usability across diverse customer bases, particularly in global retail markets. This also enhances customer satisfaction by allowing users to interact in their native language.
- **Payment Tracking Integration:** Linking the chatbot with payment gateways will allow real-time tracking of transaction statuses, including successful payments, pending verifications, and refunds. This reduces dependency on manual support and strengthens customer trust.
- **Cancellation Reasons Logging:** Incorporating a feature to capture structured cancellation reasons will provide retailers with actionable insights. This can inform business decisions related to product quality, delivery logistics, and customer expectations.
- **Multiple Orders per Login ID (Phone Number):** Enhancing the system to manage multiple concurrent orders for a single user ID will make the chatbot more practical for repeat buyers. This ensures that users can track, modify, or cancel different orders seamlessly.
- **Photo Upload Functionality:** Allowing customers to upload product images (e.g., damaged items for return) will significantly reduce resolution time and strengthen the authenticity of customer claims. This feature can also support fraud prevention in refund workflows.

- **Large scale deployment:** The project, although designed with the potential for large-scale deployment, could not be fully achieved due to financial and infrastructural constraints. This limitation underscores the need for additional investment and resource allocation in order to achieve scalability and ensure seamless deployment in real-world retail environments.

6.2. Comparative Summary of Future Features

Feature	Current Status	Proposed Enhancement	Significance
Voice Assistance	Limited to text & quick actions	Add speech input/output	Improves accessibility & user engagement
Multi-Lingual Support	English only	Support multiple languages	Expands global usability
Payment Tracking	Not available	Integrate with payment gateways	Enhances trust & reduces manual inquiries
Cancellation Reasons	Not captured	Structured reason logging	Provides insights for business improvement
Multiple Orders per Login	Supports single order	Manage multiple orders per user	Meets needs of frequent buyers
Photo Upload	Not available	Enable image attachment in chat	Supports faster dispute resolution

Table 4: Comparison of Proposed Future Features

6.3. Conclusion

The results demonstrate that the GPT-powered retail chatbot is an effective and reliable solution for enhancing customer support within retail environments. It has proven capable of delivering accurate, consistent, and context-aware assistance while integrating seamlessly with CRM systems for efficient order management and analytics. Nonetheless, constraints such as scalability under peak loads, operational costs linked to GPT API usage, and the limited functional scope indicate that there is still room for further enhancement. Addressing these limitations through the adoption of advanced features—including voice assistance for accessibility, multi-lingual capability for global inclusivity, payment tracking for transparency, structured cancellation reasons for business insights, multiple order management for repeat buyers, and photo upload for quicker dispute resolution—will strengthen both customer satisfaction and operational efficiency. Collectively, these improvements position the chatbot not only as a current solution but also as a scalable, adaptable, and future-ready platform for retail customer engagement.

Chapter – 7

Reference

1. Arce-Urriza, M., Chocarro, R., Cortiñas, M., & Marcos-Matás, G. (2025). From familiarity to acceptance: The impact of Generative Artificial Intelligence on consumer adoption of retail chatbots. Journal of Retailing and Consumer Services, 84, Article 104234. https://doi.org/10.1016/j.jretconser.2025.104234
2. Aslam, U. (2023). Understanding the usability of retail fashion brand chatbots: Evidence from customer expectations and experiences. Journal of Retailing and Consumer Services, 74, Article 103377. https://doi.org/10.1016/j.jretconser.2023.103377
3. Bass, L., Clements, P., & Kazman, R. (2021). Software architecture in practice (4th ed.). Addison-Wesley.
4. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. Advances in Neural Information Processing Systems, 33, 1877–1901.
5. Cavoukian, A. (2011). Privacy by design: The 7 foundational principles. Information and Privacy Commissioner of Ontario, Canada.
6. Chen, H., Chiang, R. H. L., & Storey, V. C. (2012). Business intelligence and analytics: From big data to big impact. MIS Quarterly, 36(4), 1165–1188.
7. Coronel, C., & Morris, S. (2016). Database systems: Design, implementation, and management (11th ed.). Cengage Learning.
8. Coronel, C., & Morris, S. (2016). Database systems: Design, implementation, & management (12th ed.). Cengage Learning.
9. Dix, A., Finlay, J., Abowd, G. D., & Beale, R. (2004). Human-computer interaction (3rd ed.). Pearson Education.
10. Dix, A., Finlay, J., Abowd, G., & Beale, R. (2004). Human-computer interaction (3rd ed.). Pearson.
11. Fowler, M. (2003). Patterns of enterprise application architecture. Addison-Wesley.
12. Garlan, D., & Shaw, M. (2017). An introduction to software architecture. In Advances in Software Engineering and Knowledge Engineering (pp. 1–40). World Scientific.
13. Grigorik, I. (2013). High performance browser networking. O'Reilly Media.

14. Jain, M., Kumar, P., Kota, R., & Patel, S. N. (2018). Evaluating and informing the design of chatbots. Proceedings of the 2018 Designing Interactive Systems Conference, 895–906. https://doi.org/10.1145/3196709.3196735
15. Jobin, A., Ienca, M., & Vayena, E. (2019). The global landscape of AI ethics guidelines. Nature Machine Intelligence, 1(9), 389–399. https://doi.org/10.1038/s42256-019-0088-2
16. Lankton, N. K., McKnight, D. H., & Tripp, J. (2015). Technology, humanness, and trust: Rethinking trust in technology. Journal of the Association for Information Systems, 16(10), 880–918.
17. Lankton, N. K., McKnight, D. H., & Tripp, J. F. (2015). Technology, humanness, and trust: Rethinking trust in technology. Journal of the Association for Information Systems, 16(10), 880–918.
18. Microsoft. (2023). Security best practices for managing secrets. Microsoft Azure Documentation. https://learn.microsoft.com/azure
19. Nielsen, J. (2012). Usability 101: Introduction to usability. Nielsen Norman Group. https://www.nngroup.com/articles/usability-101-introduction-to-usability/
20. Nielsen, J. (2012). Usability 101: Introduction to usability. Nielsen Norman Group. https://www.nngroup.com/articles/usability-101-introduction-to-usability/
21. Provost, F., & Fawcett, T. (2013). Data science for business. O'Reilly Media.
22. Richardson, C. (2018). Microservices patterns: With examples in Java. Manning Publications.
23. Shawar, B. A., & Atwell, E. (2007). Chatbots: Are they really useful? LDV Forum, 22(1), 29–49.
24. Shawar, B. A., & Atwell, E. (2007). Chatbots: Are they really useful? LDV Forum, 22(1), 29–49.
25. Sommerville, I. (2016). Software engineering (10th ed.). Pearson.
26. Tiangolo, S. (2020). FastAPI documentation. https://fastapi.tiangolo.com/
27. Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to build high-performance network programs. IEEE Internet Computing, 14(6), 80–83.
28. W3C. (2018). Web Content Accessibility Guidelines (WCAG) 2.1. World Wide Web Consortium. https://www.w3.org/TR/WCAG21/

Appendices

- ***Project recording Link*** - **Retail- Chatbot Recording**

Ch.4 *Functional Modules*

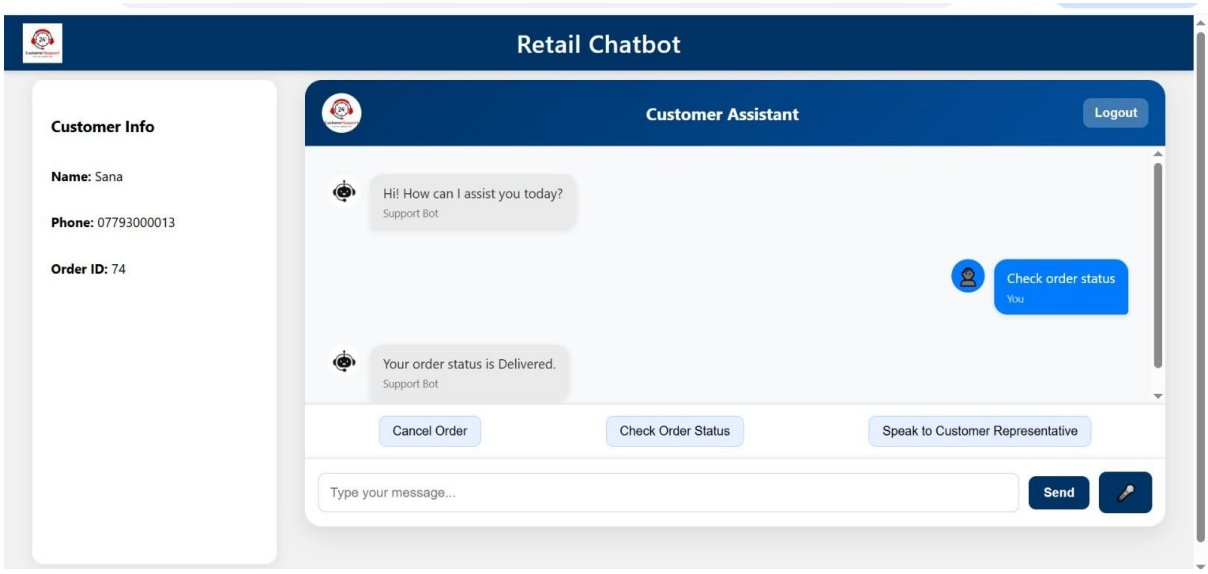


Figure 21 Order Status Enquiry.

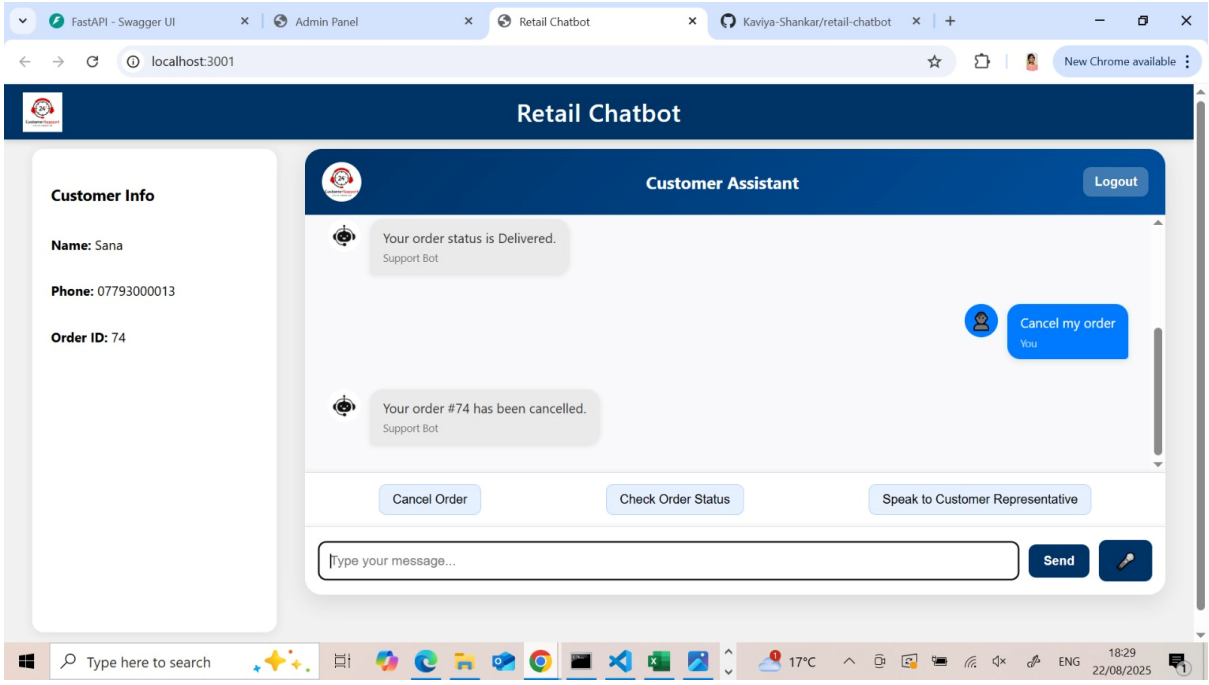


Figure 22 Order Cancellation

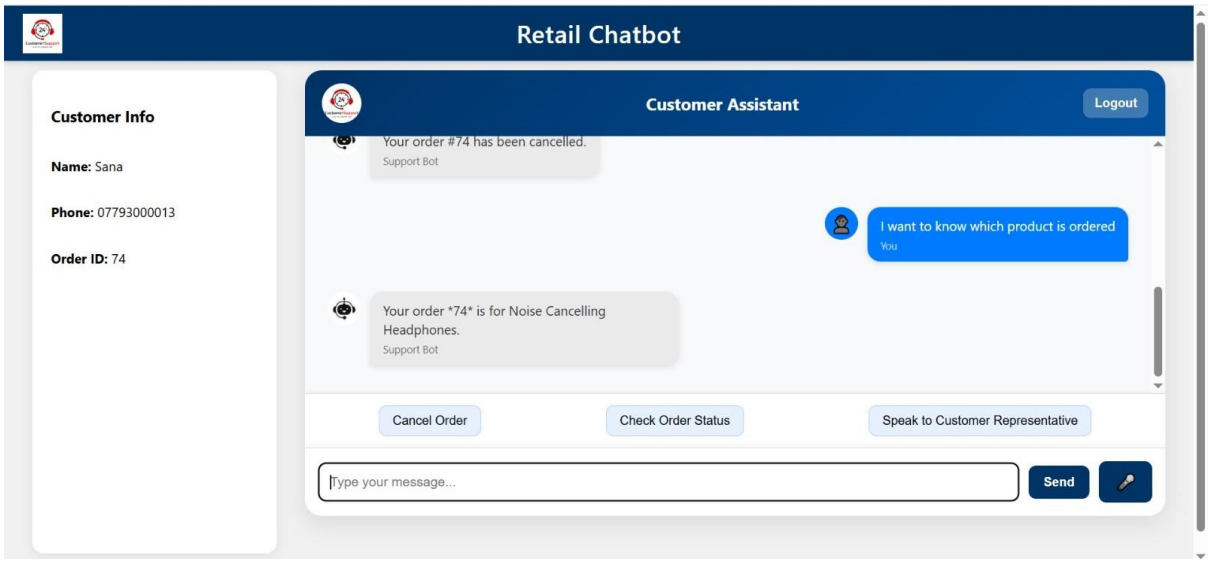


Figure 23 Product Inquiry

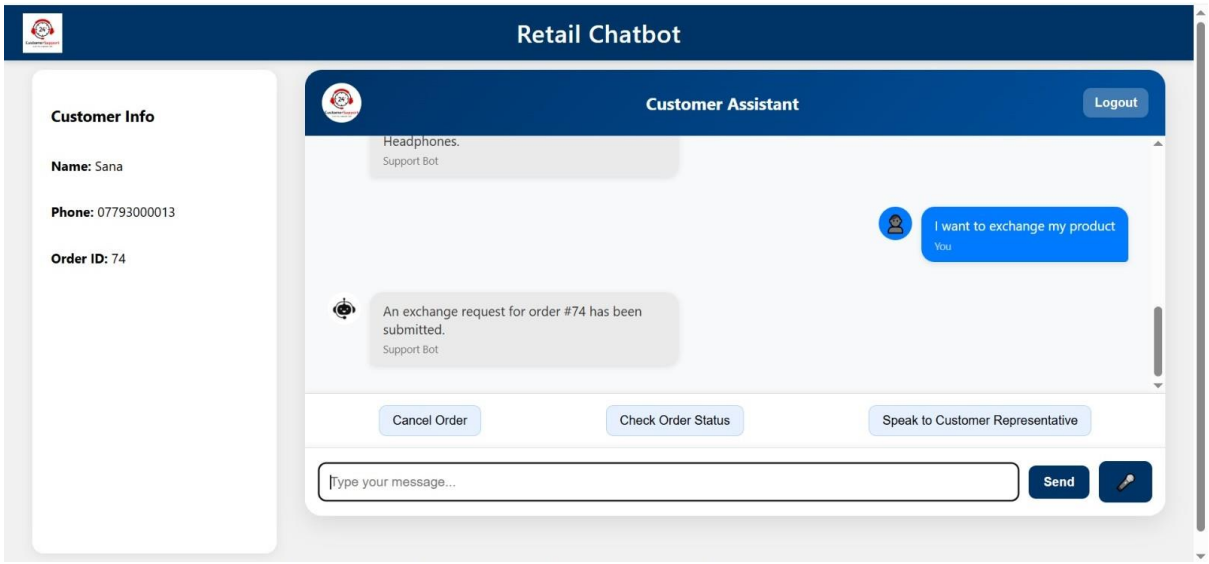


Figure 24 Exchange Request

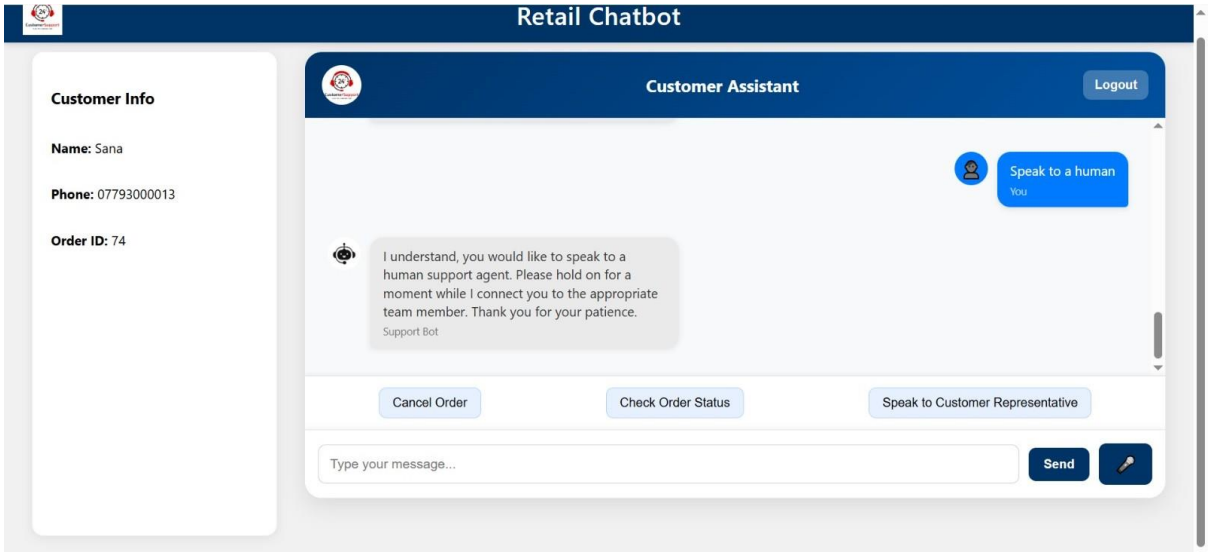


Figure 25 Customer Representative Request

Ch.4 Rationale for OpenAI Integration

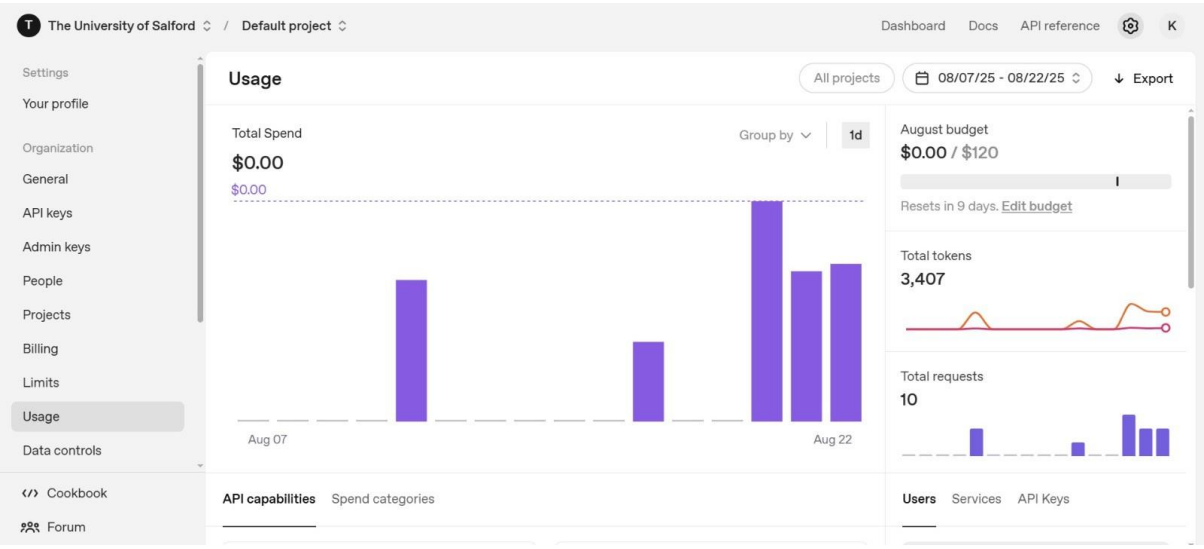


Figure 26: Usage Tracking.

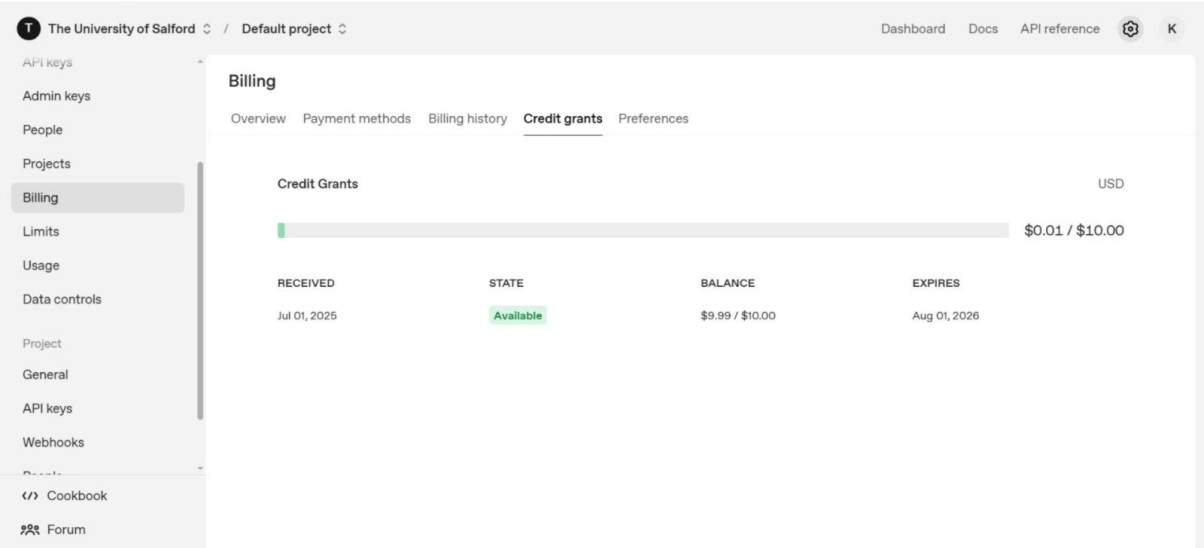


Figure 27: Purchase Credit

Ch.4. Admin Panel

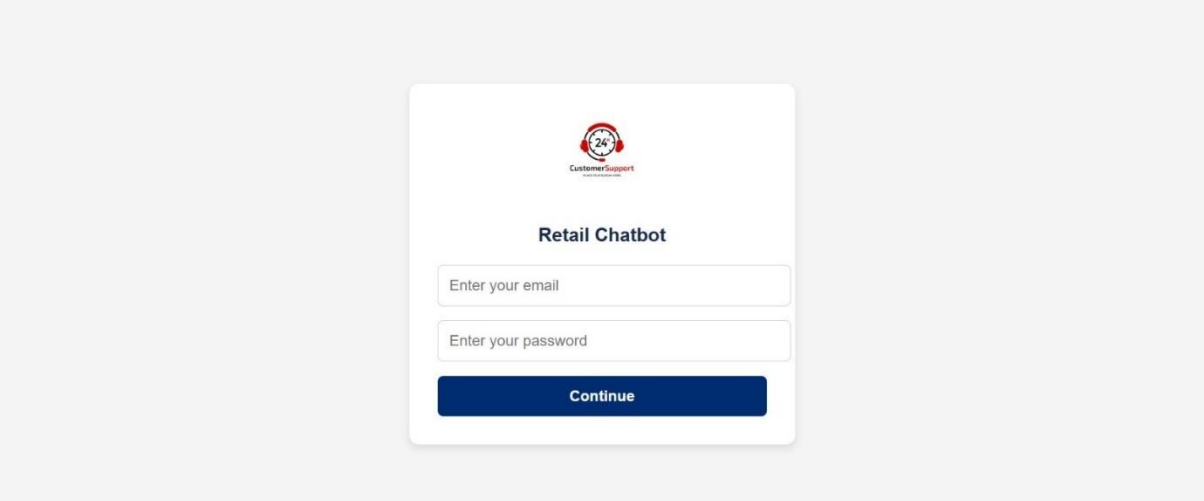


Figure 28: Admin Login Page

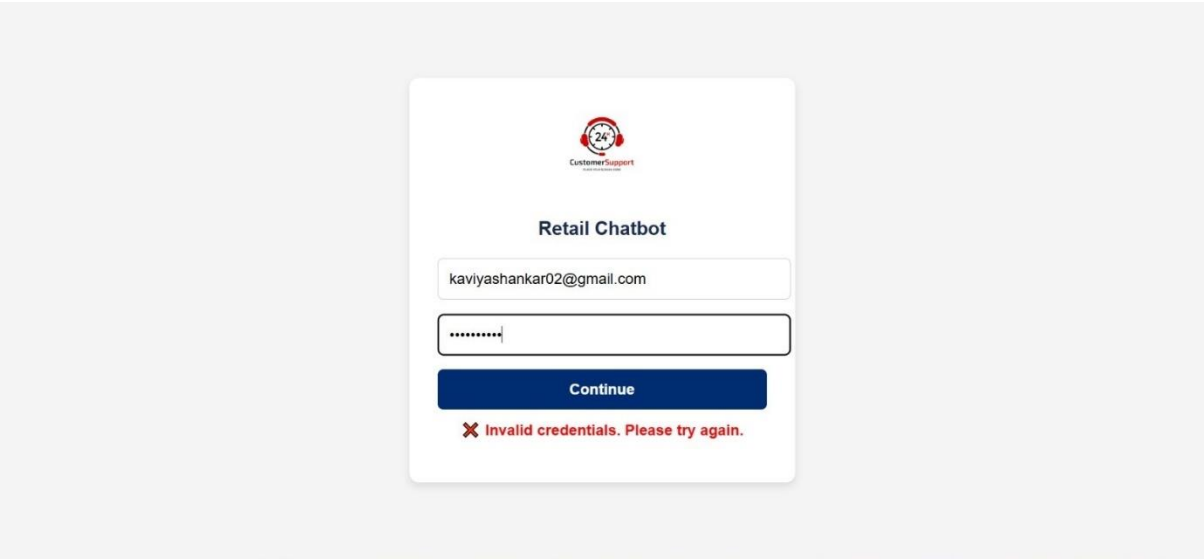
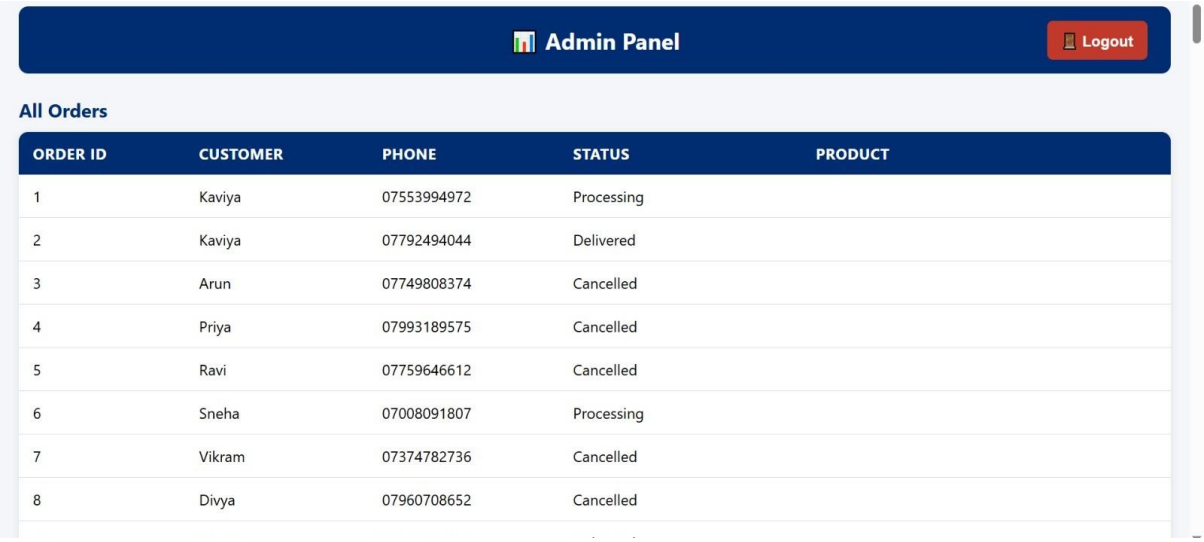


Figure 29: Admin Credentials Validations



ORDER ID	CUSTOMER	PHONE	STATUS	PRODUCT
1	Kaviya	07553994972	Processing	
2	Kaviya	07792494044	Delivered	
3	Arun	07749808374	Cancelled	
4	Priya	07993189575	Cancelled	
5	Ravi	07759646612	Cancelled	
6	Sneha	07008091807	Processing	
7	Vikram	07374782736	Cancelled	
8	Divya	07960708652	Cancelled	

Figure 30: Admin Banner

Figure 31: Navigation Bar

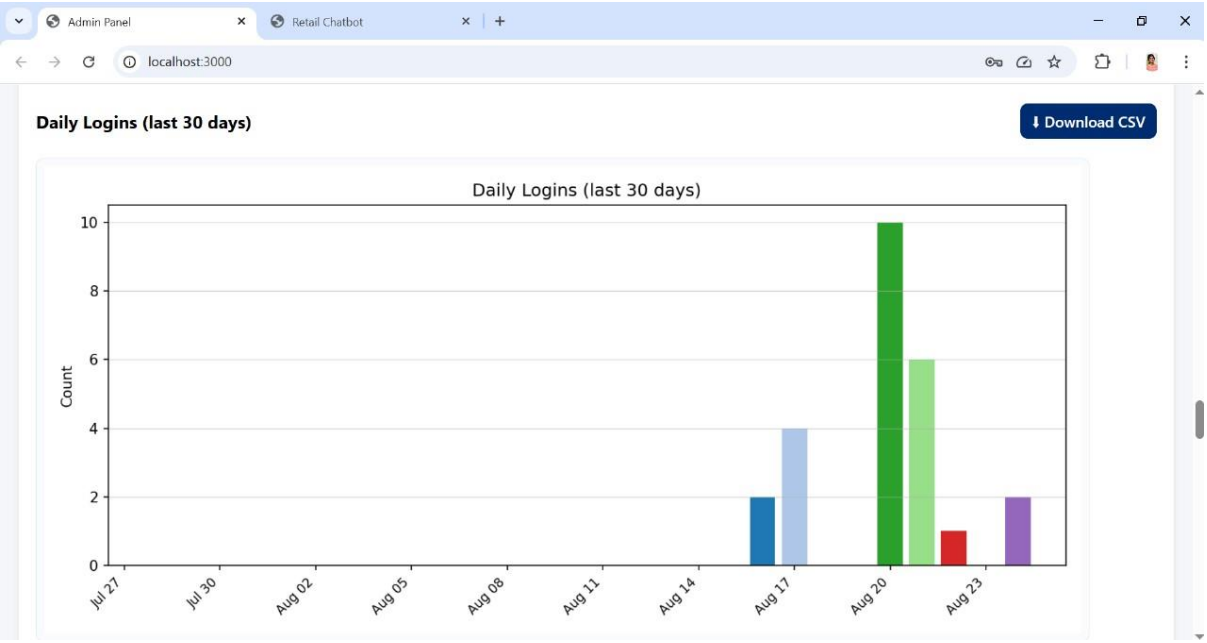


Figure 32: Login Bar Chart

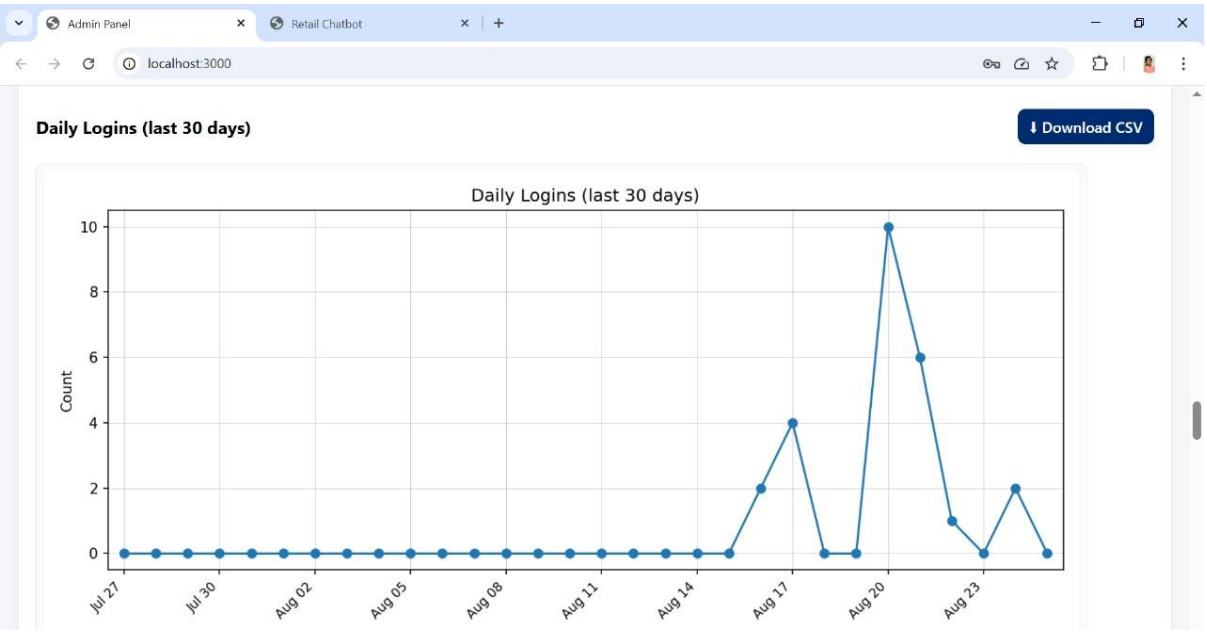


Figure 33: Login Line Chart

Logins Heatmap (last 30 days)

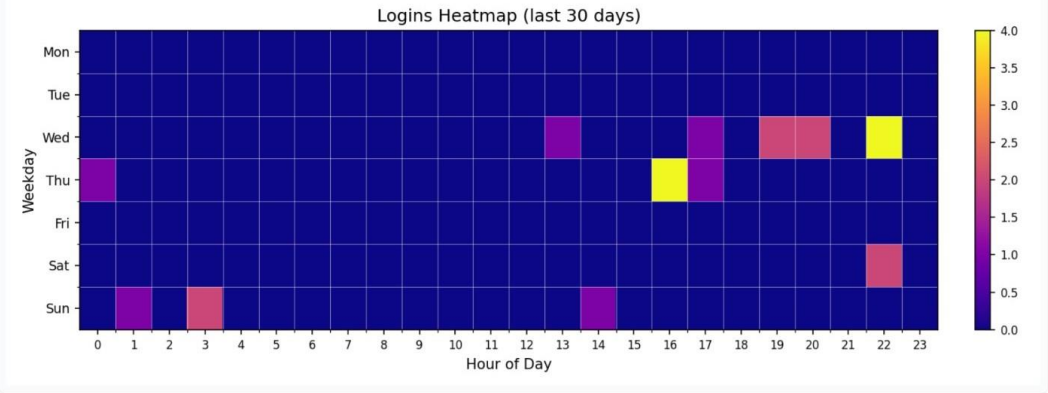


Figure 34: Logins Heatmap

IPs

IP	COUNT
127.0.0.1	22

Countries

COUNTRY	COUNT
Local Network	20
(unknown)	2

Figure 35: IPs & Countries

Phone Login Count

PHONE	COUNT
07793000017	5
07793000018	3
07793000012	2
07793000014	2
07793000016	1
07793000019	1
07793000020	1
0793000001	1
07553994972	1
07778110270	1
07792000002	1
07792000006	1
07792000008	1

Figure 36: Phone Login Counts

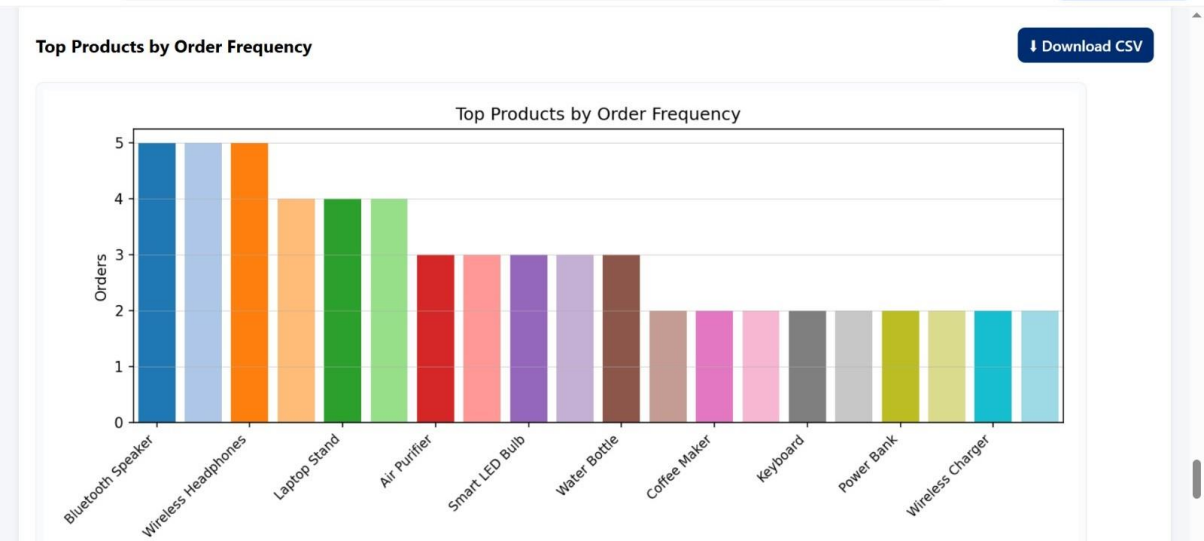
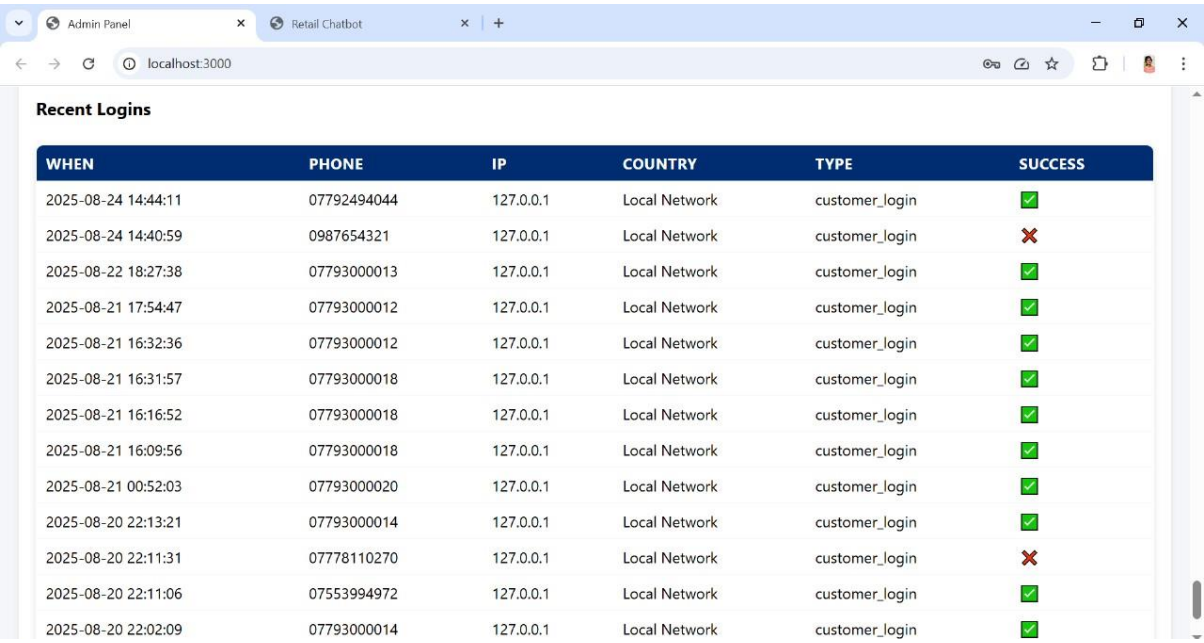
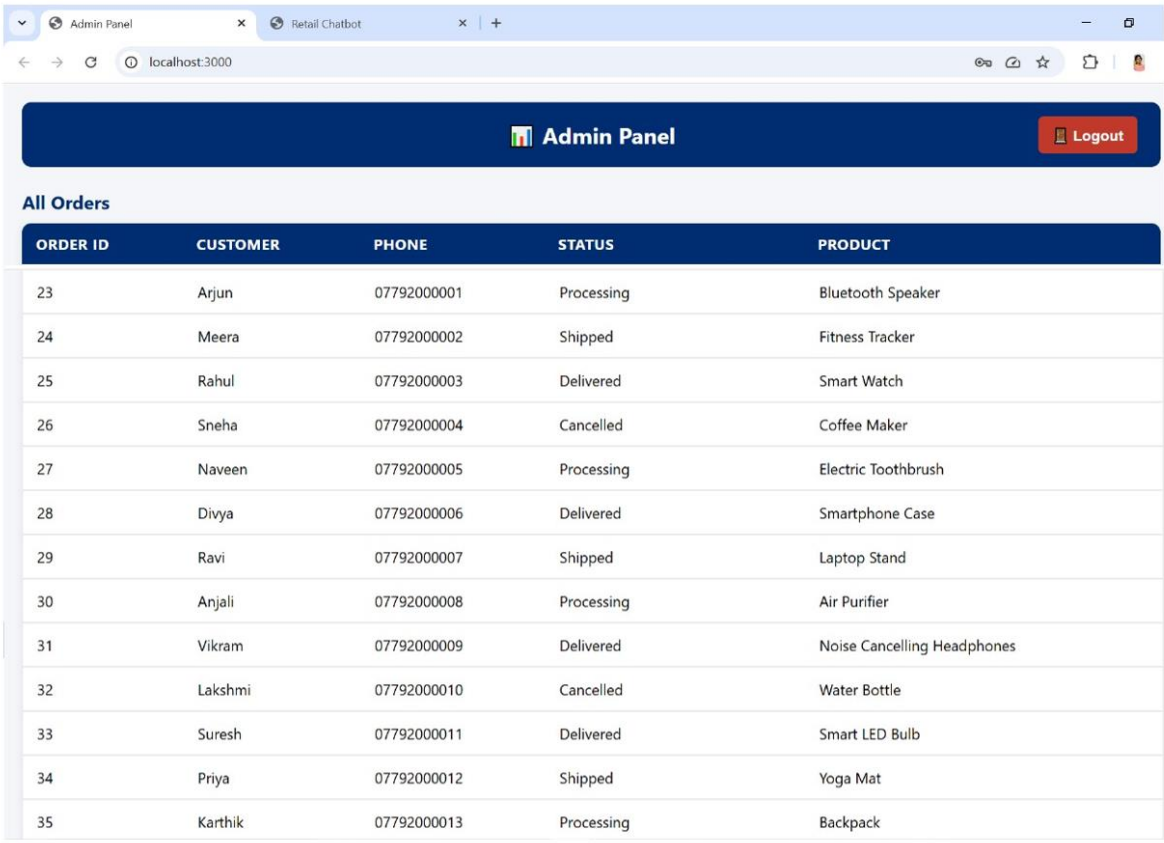


Figure 37: Top Products by Order Frequency



WHEN	PHONE	IP	COUNTRY	TYPE	SUCCESS
2025-08-24 14:44:11	07792494044	127.0.0.1	Local Network	customer_login	✅
2025-08-24 14:40:59	0987654321	127.0.0.1	Local Network	customer_login	❌
2025-08-22 18:27:38	07793000013	127.0.0.1	Local Network	customer_login	✅
2025-08-21 17:54:47	07793000012	127.0.0.1	Local Network	customer_login	✅
2025-08-21 16:32:36	07793000012	127.0.0.1	Local Network	customer_login	✅
2025-08-21 16:31:57	07793000018	127.0.0.1	Local Network	customer_login	✅
2025-08-21 16:16:52	07793000018	127.0.0.1	Local Network	customer_login	✅
2025-08-21 16:09:56	07793000018	127.0.0.1	Local Network	customer_login	✅
2025-08-21 00:52:03	07793000020	127.0.0.1	Local Network	customer_login	✅
2025-08-20 22:13:21	07793000014	127.0.0.1	Local Network	customer_login	✅
2025-08-20 22:11:31	07778110270	127.0.0.1	Local Network	customer_login	❌
2025-08-20 22:11:06	07553994972	127.0.0.1	Local Network	customer_login	✅
2025-08-20 22:02:09	07793000014	127.0.0.1	Local Network	customer_login	✅

Figure 38: Recent Logins



ORDER ID	CUSTOMER	PHONE	STATUS	PRODUCT
23	Arjun	07792000001	Processing	Bluetooth Speaker
24	Meera	07792000002	Shipped	Fitness Tracker
25	Rahul	07792000003	Delivered	Smart Watch
26	Sneha	07792000004	Cancelled	Coffee Maker
27	Naveen	07792000005	Processing	Electric Toothbrush
28	Divya	07792000006	Delivered	Smartphone Case
29	Ravi	07792000007	Shipped	Laptop Stand
30	Anjali	07792000008	Processing	Air Purifier
31	Vikram	07792000009	Delivered	Noise Cancelling Headphones
32	Lakshmi	07792000010	Cancelled	Water Bottle
33	Suresh	07792000011	Delivered	Smart LED Bulb
34	Priya	07792000012	Shipped	Yoga Mat
35	Karthik	07792000013	Processing	Backpack

Figure 39: All Orders